



# A novel software defect prediction model using two-phase grey wolf optimisation for feature selection

Ruchika Malhotra<sup>1</sup> · Kishwar Khan<sup>1</sup>

Received: 8 March 2024 / Revised: 13 May 2024 / Accepted: 23 May 2024 / Published online: 8 June 2024  
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

## Abstract

The process of accurately predicting software defects is highly crucial during the early period of software development before testing activities begin. A variety of computational methods have been constructed to achieve this based on static code metrics. However, one of the major issues in predictive modelling is the presence of redundant and irrelevant features in available datasets, which can lead to inaccuracies in the prediction model. Swarm optimization methods have shown excellent performance in Feature Selection (FS) issue mitigation and reduced the execution time of the prediction model. This study proposes a novel model for predicting software defects. This model utilizes a variant of Grey Wolf Optimiser as a wrapper-based feature selection method, paired with Synthetic Minority Oversampling Technique to balance the dataset, with the objective of maximizing the prediction efficiency of the learning model. The performance of the proposed model is assessed on 27 open-source datasets. The result findings show that the feature selection method improves prediction performance. Furthermore, the two-phase Grey Wolf Optimization-based feature selection with Random Forest classifier demonstrates superior efficacy on datasets compared to another benchmark model in handling the problem of FS. The results are also validated using statistical techniques.

**Keywords** Software defect prediction · Feature selection · Grey wolf optimisation · Synthetic minority oversampling technique (SMOTE) · Machine learning

## 1 Introduction

Software quality is a significant aspect of software engineering that needs to be considered when developing software products. The extent to which software conforms to customer needs and expectations is the software quality [1]. High-quality software products are more likely to meet user needs and expectations, resulting in greater user satisfaction. High-quality software is effective and efficient, with minimal defects. This can result in smoother business processes and better use of resources. Quality software development can reduce overall testing and maintenance efforts, resulting in lower costs. Reducing the number of

faults through quality development practises might ultimately save money because correcting defects can be expensive. High-quality software can give organizations a competitive advantage in the marketplace. Customers are more likely to choose a software product that meets their needs and performs well. Standardizing software quality practices can improve overall software quality and promote consistency across organizations [2].

Additionally, by predicting software defects, software engineers can improve their overall development processes, as they can use the data generated by Software Defect Prediction (SDP) to identify parts of the code that are prone to defects and make changes to reduce the risk of future defects data [3]. SDP is the process of identifying those software components that are most likely to have defects based on current software metrics or features for prospective software releases [4]. Software characteristics may indicate software properties like complexity and the number of operators, and the connections between the features and the class may change. Some of these traits

---

✉ Kishwar Khan  
kishwarkhan037@gmail.com  
Ruchika Malhotra  
ruchikamalhotra@dtu.ac.in

<sup>1</sup> Department of Software Engineering, Delhi Technological University, Delhi, India

might be more relevant to the class than others, while some of them might be redundant or not necessary. With Feature Selection (FS), the high-dimensional feature set can yield high correlation features. FS may select only those elements that are essential to the class. Including FS in SDP could thereby boost its performance [5]. The problems of multicollinearity and the “curse of dimensionality” are avoided by only using a few features. The total count of features present in the given dataset can be decreased with the use of FS and reduction techniques.

The overall performance of ML classifiers will suffer from high data dimensionality and the presence of redundant, noisy, and irrelevant data. These features may be selected to decrease dimensionality and improve ML algorithm performance. Additionally, FS helps researchers and programmers comprehend the data on its own and focuses on the most significant and relevant features. The performance of SDP systems can be improved by FS algorithms, according to several research publications in the literature.

Dealing with huge datasets can be challenging for the classification process due to their high dimensionality. Because of this, utilizing the most useful features possible while also optimizing classification accuracy, FS pre-processing is a must for lowering the dimensionality of datasets. The FS for classification issues based on wrapper techniques is addressed in this study using a novel Grey Wolf Optimizer (GWO) procedure, including two phases of mutation [6].

The most useful features from the actual dataset can be chosen using a variety of FS techniques. Wrapper, filter, and embedded FS techniques fall within this category. Wrapper methods evaluate the chosen subset of characteristics using a classification procedure.

The filter approach assesses the chosen features according to the different aspects of the dataset. The filter methods are more rapid than the wrapper approach because the wrapper approach serves no purpose in assessing the classifier accuracy. As opposed to the filter technique, the improved set of features which correspond with the classifier may be obtained through the wrapper technique. Low computational cost is a benefit of hybrid approaches; however, they are often machine learning-specific. The two objectives that FS must accomplish are reducing the size of the feature pool and greater accuracy in classification.

Finding an ideal subset of features is crucial in FS [7]. By analyzing the whole collection of features, the exhaustive search can create all potential subsets. Due to the fact that there are  $2^M$  subsets of features for a dataset with  $M$  features, this method is unfeasible for big datasets and has an extraordinarily high computational cost. In the last two decades, metaheuristics approaches have proven to be productive and effective in resolving complex and large-

scale issues in applications based on Machine Learning (ML) and data mining [8]. Metaheuristics drawn from nature comprise the majority of nature-inspired algorithms. Two contrasting factors are frequently used when employing these algorithms: intensification (using the most effective solutions found) and diversity (investigation of the search area). Swarm algorithms have recently been proposed and demonstrated to perform admirably for FS problems. Most of the metaheuristic algorithms suffer from local optimum, lack of diversity of the search, and imbalance between the explorative or the exploitive capabilities of the algorithm [6].

Given the challenges that can hinder effective feature selection, we were motivated to develop a method that leverages the Grey Wolf Optimizer algorithm GWO. This approach addresses the issues associated with feature selection and aims to find more efficient solutions. The Grey Wolf Optimizer algorithm has demonstrated notable success across various applications, making it a strong candidate for this purpose.

To address the issues of local optima and limited diversity, the proposed algorithm incorporates a two-phase mutation operation (2M-GWO) to enhance its performance. This mutation operation is designed to improve the algorithm's exploitation capabilities, resulting in more effective and diverse solutions.

The following are the key contributions of our work:

- We propose a novel prediction model based on the 2M-GWO algorithm and five ML classifiers for SDP, which will select the most promising features and enhance the predictive capability of the prediction model. To the best of our knowledge, this model has never been employed for SDP.
- The performance of the proposed SDP model is assessed using 27 publicly available datasets to ensure the generalizability of the results. The investigation aimed to determine the impact of the improvements in 2M-GWO on optimization performance and robustness.
- A number of comparisons are made on the Receiver Operating Characteristic Area Under Curve (ROC\_AUC) value using statistical concepts such as mean value and Friedman test. These methods offer a robust assessment of algorithm performance across different scenarios, providing insight into average ROC\_AUC results and identifying significant variations among the compared algorithms.
- The proposed model based on the 2M-GWO algorithm is evaluated against five other metaheuristics techniques (Harris Hawks Optimization (HHO), Salp Swarm Optimization (SSO), Whale Optimization (WO), Jaya Optimization (JO), and Sine Cosine Optimization

(SCO)), and experimental results demonstrated its superiority over competing algorithms.

The organization of the research paper is as outlined: The review of the past literature is covered in Sect. 2, which discusses the various methodologies that have been used for SDP using evolutionary FS methods. In Sect. 3, the proposed SDP model based on 2M-GWO for FS is described. Section 4 covers the simulated results of the proposed model, along with qualitative and comparative analysis using statistical tools. Finally, Sect. 5 contains the conclusion and suggestions for future work.

## 2 Related work

The effectiveness of SDP models was examined in recent studies to determine how evolutionary FS techniques affected their performance. Table 1 presents a detailed description of existing literature related to the use of metaheuristic techniques in SDP.

## 3 Proposed methodology

To solve the SDP issue, we developed a wrapper strategy based on an enhanced version of the GWO algorithm. Figure 1 explores the mechanism of the suggested algorithm. A tenfold cross-validation (10-CV) procedure is performed at each iteration. Initially, the algorithm applies GWO on the SDP dataset. Using the 10-CV method, the chosen dataset is subdivided at each iteration. The SMOTE mechanism is used to resolve the unbalanced problem of the dataset on which the model should be trained. This procedure will generate a training dataset using oversampling. Five distinct classifiers (i.e., SVM, RF, GB, AB, and KNN) are used to construct an SDP model, which is then assessed using K-Fold = 10. The suggested method terminates whenever the best solution in terms of the objective function is attained, or the value of iterations reaches its maximum value. The model is shown in the following points.

### 3.1 Dataset

Defects are faults or errors that have been made in the past when developing software systems. These inaccuracies are used to forecast the likelihood of a fault or the total number of defects. Statistics on software defects can clearly show which software modules or components are particularly susceptible to faults. These undesirable defects can occur at many software systems levels and in a variety of granularities. Software metrics and developer information make

up the majority of the data in software defect databases. Software attributes may be divided into process, product, and requirements attributes.

For this study's defect datasets, data from four different sources were collected (NASA, PROMISE, RELINK, and AEEEM). This study made use of the NASA dataset as it was created by Shepperd, et al. The NASA datasets include metrics generated from static code parts that take into account the size and complexity of the code [9]. Projects from the PROMISE repository are based on features that are gathered at the class level for object-oriented software and information about defects found in software components. This dataset was developed using the JAVA programming language and was retrieved from the Apache software [10]. The open-source RELINK dataset comprises three open-source projects (Apache HTTP, Safe, and Zxing) and has a total of twenty-six attributes [11].

AEEEM datasets differ from other data [12] as they include software attributes from source code measurements. The change metrics, entropy, and churn of source code metrics form the foundation of source code metrics. These datasets are obviously dependent on various software features. The datasets details are mentioned in Table 1.

### 3.2 Feature selection using 2M-GWO

#### 3.2.1 Grey wolf algorithm (GWO)

Grey wolves (GWs) are superb predators with exceptional hunting abilities. This is due to the fact that they live in a disciplined and orderly pack. The GWO algorithm was recently presented to imitate the hunting, seeking, and surrounding behaviour of GWs [13]. There are four main varieties of GWs with varying levels of dominance and leadership in the wolf population (Table 2). The first, second, and third-best solutions in the GWO method are denoted by the letters  $\alpha$ ,  $\beta$  and  $\delta$ , correspondingly. The rest of the possible solutions are referred to as  $\omega$ .

The hunting mechanism in GWO involves three main steps: encircling the prey, hunting (attacking), and searching for prey. These steps are mathematically modelled as follows:

**Encircling Prey** The wolves encircle the prey during the hunt. This behaviour is modelled using the position vectors of the prey and the wolves. The position of each wolf is updated according to the position of the prey using the following equations:

$$D_v = |C_v \cdot X_v p - X_v(t)| \quad (1)$$

$$X_v(t+1) = X_v p(t) - A_v \cdot D_v \quad (2)$$

**Table 1** Detailed description of existing related work

| References | Dataset                                                                                                                                                                                                 | Techniques                                                                                                                                                                                                                                                | Benefits                                                                                                                                                                                                                                                                                                 | Limitations                                                                                                                                                                                      |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [30]       | PROMISE Dataset:<br>Ant (1.7),<br>Camel (1.2, 1.4, 1.6),<br>Jedit (3.2, 4.0 4.1, 4.2),<br>Lucene (2.0, 2.2, 2.4),<br>Log4j (1.0, 1.1, 1.2),<br>Xalan (2.4, 2.5, 2.6)                                    | Levy Sparrow Search (LSS),<br>weighted dual cross<br>recurrent network                                                                                                                                                                                    | The DCRN is used for its ability<br>to handle complex data<br>relationships, while the LSS<br>algorithm helps in optimizing<br>the model's parameters,<br>enhancing the model's ability<br>to predict faults accurately                                                                                  | The study only utilizes<br>PROMISE datasets for<br>validation. this could limit the<br>understanding of the model's<br>robustness and effectiveness<br>across a broader range of<br>applications |
| [31]       | PROMISE Dataset (19<br>projects)                                                                                                                                                                        | Mutation Boosted Salp Swarm<br>Optimizer (MBSSO) Rough<br>Set The<br>ory, Kernel Extreme Learning<br>Machine (KELM)                                                                                                                                       | This study allows for effective<br>feature selection based on<br>relevance and dependency,<br>leading to improved defect<br>detection models with higher<br>accuracy and efficiency                                                                                                                      | The study faces disadvantages<br>such as complexity and high<br>computational costs due to the<br>integration of MBSSO with<br>Rough Set Theory                                                  |
| [32]       | PROMISE Dataset:<br>KC (1, 3)<br>PC (1, 2, 4, 5),<br>MC1, JM1, MW1, CM1,<br>MC2                                                                                                                         | Golden Jackal Optimization,<br>K-Nearest Neighbor, Decision<br>Tree, Quadrative Discriminant<br>Analysis and Naive Bayes                                                                                                                                  | The study presents a novel and<br>effective feature selection<br>method, FSGJO, that can<br>significantly improve the<br>performance of software fault<br>prediction models by<br>identifying the most relevant<br>features                                                                              | The study has limitations, such<br>as limited generalizability, as<br>the effectiveness of the FSGJO<br>algorithm may be specific to<br>the datasets used in the<br>research                     |
| [24]       | RELINK Dataset (3<br>projects),<br>AEEEM Dataset (5<br>projects)                                                                                                                                        | MIC_SM_FS (utilizing maximal<br>information coefficient and<br>statistical measures) and<br>BPSO_FS (using binary<br>particle swarm optimization),<br>Naïve Bayes (NB), Random<br>Forest (RF), and Logistic<br>Regression (LR), TCA +                     | MIC_SM_FS selected features<br>with high importance and<br>similar distribution to the<br>target project using maximal<br>information coefficient and<br>statistical measures. BPSO_FS<br>optimized performance by<br>selecting representative<br>features through binary<br>particle swarm optimization | If the source data is not<br>representative or if it contains<br>biases, the predictions made<br>for the target projects might<br>not be accurate                                                |
| [33]       | NASA Dataset:<br>PC1, CM1, MC2, MW1<br>AEEEM Dataset:<br>JDT, PDE, ML<br>MORPH Dataset:<br>ant-1.3, arc, camel-1.0,<br>activemq-5.0., hbase-<br>0.94.0, hive-0.9.0,<br>groovy-1_6_BETA_1,<br>jruby-1.1) | Sparrow Search algorithm,<br>Extreme Learning Machine                                                                                                                                                                                                     | The study presents a<br>comprehensive approach to<br>improving software defect<br>prediction through a novel<br>optimization algorithm and an<br>ensemble learning framework,<br>addressing key challenges<br>such as parameter<br>optimization, data imbalance,<br>and model generalization             | The scalability of the proposed<br>algorithm in handling very<br>large datasets is not discussed                                                                                                 |
| [34]       | PROMISE Dataset: Ant<br>1.7,<br>Camel (1.0, 1.2, 1.4,1.6),<br>Jedit (3.2, 4.0, 4.1, 4.2,<br>4.3),<br>Log4j (1.0, 1.1), Lucene<br>2.0,<br>Xalan (2.4, 2.5, 2.6)                                          | Enhanced Binary Whale<br>Optimization Algorithm,<br>K-Nearest Neighbors (KNN),<br>Naive Bayes (NB),<br>Linear Discriminant Analysis<br>(LDA), Linear Regression<br>(LR), Decision Tree (DT),<br>Support Vector Machine<br>(SVM), and<br>RandomForest (RF) | The enhanced version of the<br>BWOA referred to as<br>improving the diversity and<br>exploration capabilities of the<br>BWOA, makes it more<br>effective in navigating the<br>feature selection search space                                                                                             | The generalization of these<br>results to other types of<br>software projects or datasets is<br>not guaranteed                                                                                   |

**Table 1** (continued)

| References | Dataset                                                                                                                             | Techniques                                                                                        | Benefits                                                                                                                                                                                                                                                                                                                                                           | Limitations                                                                                                                                                                                                                    |
|------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [35]       | PROMISE Dataset: PC1, KC2, KC1, JM1, and CM1                                                                                        | War Strategy Optimization (WSO), Kernel-based Extreme Learning Machine (KELM)                     | The study reports an average increase of over 90% in accuracy across various parameters, which underscores the effectiveness of the combined approach in identifying defects in software components                                                                                                                                                                | The advanced algorithms used, such as WSO and KELM, may require substantial computational resources for training and optimization, especially when dealing with large datasets                                                 |
| [36]       | PROMISE Dataset: Ar (1, 3, 4, 5, 6), Ant 1.7, CM1, Jedit (4.0, 4.1), Tomcat 6<br>NASA Dataset: JM1, MW1, KC (1, 2, 3), PC (1, 2, 4) | Mine Blast Algorithm and Simulated Annealing (MBA-SA), back propagation neural network (BPNN)     | This hybrid approach aims to optimize the input connection weights and biases for a backpropagation neural network, significantly improving prediction accuracy across multiple datasets compared to existing methods                                                                                                                                              | The suggested model may need significant computational resources for training and optimization                                                                                                                                 |
| [37]       | AEEEM Dataset: JDT, PDE, EQ                                                                                                         | Bat Optimisation (BA), Improved Fisher's Linear Discriminant, Enhanced Deep Neural Network (EDNN) | This approach effectively reduces the dimensionality of the data, which in turn enhances the performance of the software bug prediction model. The IFLD method helps in selecting the most discriminative features from the software modules, which are crucial for accurate bug prediction                                                                        | The study is limited to only the AEEEM dataset                                                                                                                                                                                 |
| [38]       | PROMISE Dataset: Lucene<br>NASA dataset: KC (1, 2), PC1, JM1                                                                        | Genetic Programming, Adaboost                                                                     | The proposed model tends to achieve superior predictive accuracy compared to simple statistical, ensemble, or individual machine-learning techniques alone                                                                                                                                                                                                         | Due to the stochastic nature of genetic algorithms and the selection of features, there can be variability in the predictive performance                                                                                       |
| [39]       | PROMISE Dataset: Camel (1.0, 1.2, 1.4, 1.6), Jedit (3.2, 4.0, 4.1,4.3), Ant-1.7, Log4j (1.1, 1.0, 1.2,), Xalan (2.4, 2.2)           | Butterfly Optimization, Random Forest, ADASYN                                                     | The use of Butterfly optimization for feature selection in the context of Ensemble Random Forest with Adaptive synthetic sampling (E-RF-ADASYN) helps in identifying the most significant features for fault prediction                                                                                                                                            | There is a risk of overfitting, where the model fits the training data too closely and fails to generalize well to unseen data                                                                                                 |
| [40]       | NASA Dataset: PC5, JM1, KC1                                                                                                         | Binary Rao optimization, Naive Bayes (NB), Support Vector Machine (SVM)                           | The proposed approach in the study is a hybrid feature selection method that combines a filter-wrapper approach with a multi-criteria decision-making (MCDM) method and the Rao optimization algorithm. This method is designed to enhance the accuracy of software defect prediction models by selecting the most informative features from software metrics data | The study mentions the use of self-adaptive mechanisms to adjust parameters like population size, but the optimal configuration of these parameters can vary widely between different datasets and feature selection scenarios |

**Table 1** (continued)

| References | Dataset                                                                                                                                                                                                                            | Techniques                                                                                                                                                         | Benefits                                                                                                                                                                                                                                                                                                                            | Limitations                                                                                                                                                                                               |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [41]       | NASA Dataset:<br>CM1, JM1, KC1                                                                                                                                                                                                     | Elitist Self-Adaptive Multi-Population Social Mimic Optimization (ESAMP-SMO)                                                                                       | This proposed model combines elements of social mimic optimization with genetic operations like crossover and mutation to enhance the search capabilities of the algorithm                                                                                                                                                          | The generalization of the method across diverse software projects with different characteristics and defect patterns is not explicitly addressed                                                          |
| [42]       | PROMISE Dataset:<br>Ant (1.6, 1.7),<br>Camel (1.4, 1.6),<br>Ivy 2.0, Jedit (4.2, 4.3),<br>Poi 2.0, Prop 6,<br>Synapse 1.2,<br>Xalan (1.2, 2.5)<br>RELINK Dataset:<br>Apache, Safe, Zxing<br>AEEEM Dataset:<br>JDT, LC, EQ, PDE, ML | Whale Optimization Algorithm (WOA) and Simulated Annealing (SA), convolutional neural network (CNN) and kernel<br>extreme learning machine (KELM)                  | This introduced model effectively selects an optimal representative feature subset by leveraging the strong local search capability of SA to enhance the exploitation performance of WOA and the strong global search capability of WOA to boost the exploration of SA simultaneously                                               | The computational complexity of combining WOA and SA could result in longer processing times, making the approach less efficient for large datasets                                                       |
| [43]       | PROMISE Dataset:<br>Ant 1.7,<br>Camel (1.2, 1.4, 1.6),<br>Jedit (3.2, 4.0, 4.1, 4.2),<br>Log4j (1.0, 1.1, 1.2),<br>Lucene (2.0, 2.2, 2.4),<br>Xalan (2.4, 2.5, 2.6)                                                                | Boosted Whale Optimization Algorithm (BWOA), Decision Tree and K-Nearest Neighbour                                                                                 | The suggested improvement facilitates the WOA in avoiding local optima by augmenting its exploration process. Through thorough evaluation, it's evident that the proposed approach significantly enhances the performance of ML classification techniques, surpassing both the original WOA and six other commonly employed methods | The study is limited to the PROMISE dataset; the limitations of this study in terms of generalizability primarily revolve around the potential lack of diversity within the algorithm's application scope |
| [44]       | PROMISE Dataset:<br>Ant-1.7,<br>Camel (1.0, 1.2, 1.4, 1.6),<br>Jedit (3.2, 4.0, 4.1, 4.2),<br>Log4j (1.0, 1.1, 1.2),<br>Xalan (2.4, 2.7)                                                                                           | Enhanced Binary Harris Hawk Optimization (EBHHO),<br>Selection<br>K-nearest neighbours (kNN),<br>Decision Trees (DT), and<br>Linear<br>Discriminant Analysis (LDA) | By reducing the dimensionality through feature selection, the algorithm improves the manageability and performance of the learning models used for predicting software faults                                                                                                                                                       | The use of only one benchmark dataset could limit the applicability of the findings to other types of datasets or real-world scenarios that have different characteristics                                |
| [29]       | PROMISE Dataset:<br>Ant 1.7,<br>Camel (1.0, 1.2, 1.4, 1.6),<br>Jedit (3.2, 4.0, 4.1, 4.2, 4.3),<br>Log4j (1.0, 1.1, 1.2),<br>Xalan-2.4, 2.7)                                                                                       | Enhanced Binary Moth Flame Optimization (EBMFO),<br>k-nearest neighbours (k-NN),<br>Decision Trees (DT), and<br>Linear discriminant analysis (LDA)                 | The suggested approach improves the results from the literature and demonstrates the significance of transfer functions for FS methods                                                                                                                                                                                              | The proposed algorithm inherently carries the computational complexity for large datasets having more than 20 features                                                                                    |



**Table 1** (continued)

| References | Dataset                                                                                                                                                       | Techniques                                                                                                                                                  | Benefits                                                                                                                                                                                                                                                                                                                            | Limitations                                                                                                                                                                                                                                         |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [45]       | AEEEM Dataset:<br>EQ,<br>MORPH Dataset:<br>Velocity 1.4,<br>SOFTLAB Dataset:<br>Ar1,<br>NASA Dataset:<br>CM1, PC2,<br>RELINK Dataset:<br>Apache               | Firefly Particle Swarm<br>Optimization, KNN, DT                                                                                                             | The firefly algorithm helps in selecting the most influential features, while particle swarm optimization fine-tunes the prediction model                                                                                                                                                                                           | The integration of the firefly algorithm with particle swarm optimization is computationally intensive, which may limit its practical applicability in environments with limited computational resources or where rapid defect prediction is needed |
| [46]       | NASA Dataset:<br>CM1, PC1, KC1, KC2                                                                                                                           | Firefly Algorithm (FA) support vector machine (SVM), Naïve Bayes (NB) as well as K-nearest neighbour (KNN)                                                  | The Firefly Algorithm (FA) algorithm is particularly noted for its simplicity, efficiency, and ability to find global optima in multimodal optimization landscapes. The outcomes of the experiment indicate that the utilization of the FA in the FS can yield a classification efficiency that surpasses that of all other methods | The algorithm's effectiveness and efficiency might vary when applied to other datasets with different characteristics or higher dimensionality, which could affect its generalization capability                                                    |
| [47]       | PROMISE Dataset: Ant-1.7,<br>Camel (1.0, 1.2, 1.4, 1.6),<br>Jedit (3.4, 4.0, 4.2, 4.3),<br>Log4j (1.0, 1.1, 1.2, 2.0, 2.2, 2.4),<br>Xalan-2.4, 2.5, 2.6, 2.7) | Binary Genetic Algorithm (BGA), Binary Particle Swarm Optimization (BPSO), Binary Ant Colony Optimization (BACO), layered recurrent neural network (L-RNN), | The layered structure helps capture complex patterns and dependencies in the data, which improves the accuracy of software fault prediction. The findings demonstrate that the suggested strategy works comparatively better than other existing approaches                                                                         | The study is limited to only PROMISE dataset                                                                                                                                                                                                        |
| [48]       | NASA Dataset:<br>PC1, PC3, CM1,<br>PROMISE Dataset: Ant-1.7, Ivy-2.0, Lucene 2.2, Tomcat, Xalan-2.4, Poi-2.0                                                  | Binary Teaching–Learning–Based Optimization (BTLBO), KNN                                                                                                    | The approach produces a balanced selection of extremely important samples that may be used to train a classification technique for accurate prediction of possibly defective software products                                                                                                                                      | The process of finding the optimal sample subset using BTLBO can be computationally expensive                                                                                                                                                       |

$$A_v = 2a_v \cdot \text{random}_{v1} - a_v \quad (3)$$

$$C_v = 2 \cdot \text{random}_{v2} \quad (4)$$

$$a = 2 - t \left( \frac{2}{\text{iterations}} \right) \quad (5)$$

Here,  $X_v(t)$  represents the position vector of a wolf at iteration  $t$ ,  $X_vp(t)$  represents the position of the prey.  $C_v$  and  $A_v$  are coefficient vectors. The parameter *iterations* determine the maximum number of iterations. The  $\text{random}_{v1}$  and  $\text{random}_{v2}$  are random vectors, whereas  $a_v$  decreases linearly from 2 to 0 over the course of iterations.

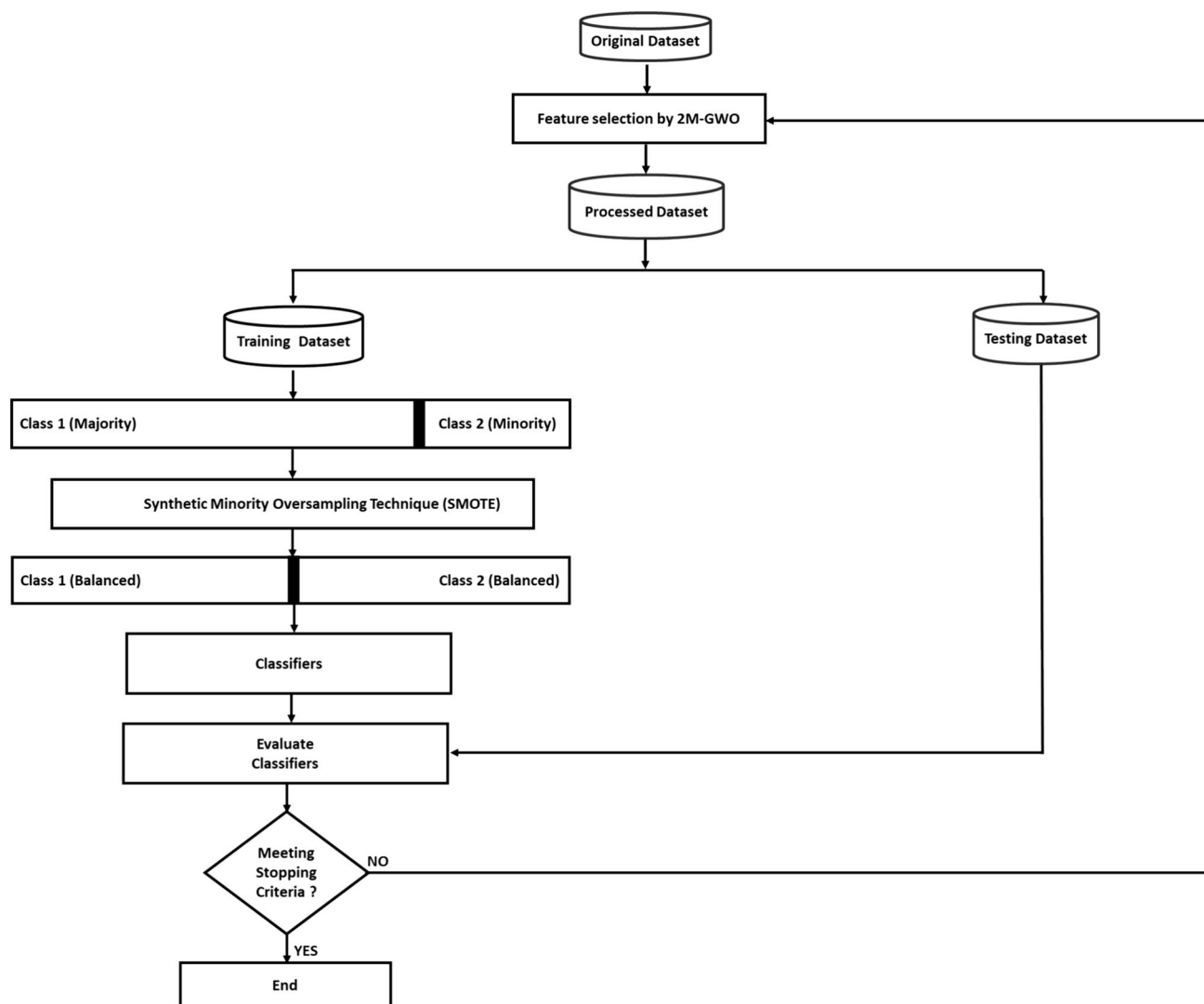
**Attacking the Prey** The alpha (best solution), beta, and delta wolves have better knowledge about the potential location of the prey. Therefore, the positions of the other

wolves (omegas) are updated based on the positions of the alpha, beta, and delta wolves:

$$X_v(t+1) = \frac{X_{v1} + X_{v2} + X_{v3}}{3} \quad (6)$$

Here  $X_{v1}$ ,  $X_{v2}$  and  $X_{v3}$  are the positions of the alpha, beta, and delta wolves, respectively, updated using similar equations to the encircling mechanism but cantered around each of these three wolves.

**Searching for Prey** To simulate the search for prey, the wolves diverge from each other and explore the search space. This is modelled by allowing the random vectors  $\text{random}_{v1}$  and  $\text{random}_{v2}$  to take values that can potentially increase the distance between the position of a wolf and the position of the prey.



**Fig. 1** Proposed methodology

Although the standard GWO is known for its effectiveness in various optimization tasks, but it faces challenges such as premature convergence and getting trapped in local optima when dealing with complex feature selection problems.

To address these challenges, the authors propose a two-phase mutation mechanism integrated into the GWO. This enhancement aims to improve the exploitation capabilities of the algorithm, making it more effective in navigating the search space of feature selection problems.

### 3.2.2 Two-phase mutation grey wolf optimisation (2M-GWO)

The Two-Phase Mutation Grey Wolf Optimizer (TMGWO) is an enhanced version of the standard GWO, specifically adapted for feature selection problems in classification

tasks. This enhancement integrates a two-phase mutation mechanism to improve the exploitation capabilities of the algorithm, making it more effective in navigating the complex search space of feature selection problems. The benefits of using two mutation phases are as follows.

**First Mutation Phase** This phase aims to reduce the number of selected features while maintaining high classification accuracy. It selectively mutates certain features of the best solution (alpha wolf), turning some of the selected features off (from 1 to 0).

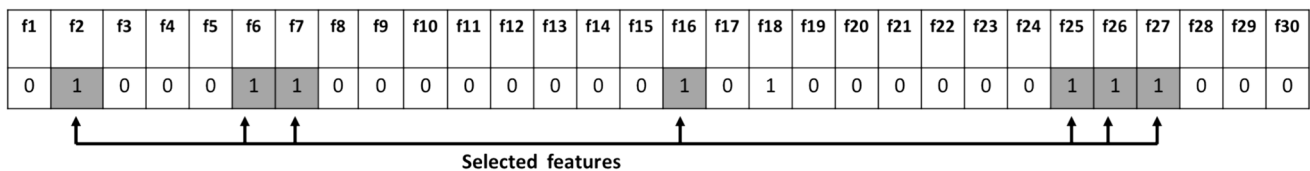
**Second Mutation Phase** This phase attempts to add potentially informative features that could increase classification accuracy. It operates by turning some unselected features on (from 0 to 1) based on their potential to improve the solution.

This 2M-GWO algorithm includes four steps: Initialization (step 1), evaluation (step 2), transformation



**Table 2** Dataset details

| Projects           | #Instances | #Features | #Defective modules | % of Defective modules |
|--------------------|------------|-----------|--------------------|------------------------|
| PROMISE-Ant-1.7    | 745        | 20        | 166                | 22.28                  |
| PROMISE-Camel-1.2  | 608        | 20        | 216                | 35.5                   |
| PROMISE-Camel-1.4  | 872        | 20        | 145                | 16.6                   |
| PROMISE-Jedit-3.2  | 272        | 20        | 90                 | 33.1                   |
| PROMISE-Jedit-4.1  | 312        | 20        | 79                 | 25.32                  |
| PROMISE-Jedit-4.2  | 367        | 20        | 48                 | 13.07                  |
| PROMISE-Log4j-1.0  | 135        | 20        | 34                 | 25.18                  |
| PROMISE-Log4j-1.1  | 109        | 20        | 37                 | 33.94                  |
| PROMISE-Lucene-2.0 | 195        | 20        | 91                 | 46.66                  |
| PROMISE-Lucene-2.2 | 247        | 20        | 144                | 58.3                   |
| PROMISE-Tomcat     | 853        | 20        | 77                 | 9.02                   |
| PROMISE-Xalan 2.4  | 723        | 20        | 110                | 15.2                   |
| PROMISE-Xalan 2.5  | 803        | 20        | 387                | 48.19                  |
| PROMISE-Xalan- 2.6 | 885        | 20        | 411                | 46.4                   |
| RELINK-Apache      | 194        | 26        | 98                 | 50.52                  |
| RELINK-Safe        | 56         | 26        | 22                 | 39.29                  |
| RELINK-Zxing       | 399        | 26        | 118                | 29.57                  |
| NASA-Cm1           | 328        | 37        | 42                 | 12.8                   |
| NASA-Mw1           | 253        | 37        | 27                 | 10.67                  |
| NASA-PC1           | 1109       | 37        | 77                 | 6.94                   |
| NASA-PC3           | 1077       | 37        | 134                | 12.44                  |
| NASA-PC4           | 1458       | 37        | 178                | 12.21                  |
| AEEM-EQ            | 325        | 61        | 129                | 39.69                  |
| AEEM-JDT           | 997        | 61        | 206                | 20.66                  |
| AEEM-LC            | 399        | 61        | 64                 | 16.04                  |
| AEEM-ML            | 1862       | 61        | 245                | 13.15                  |
| AEEM-PDE           | 1492       | 61        | 209                | 14                     |

**Fig. 2** Solution representation of 2M-GWO

function (step 3), and two-phase mutation (step 4) make up the core components of 2M-GWO.

**Initialization** The first step is to initialize a colony of “n” GWs or search agents (SA) generated randomly. Each SA represents a potential solution and has a size of “d” equal to the number of attributes in the actual dataset [14]. For classifying classes, the FS is used to identify the subset of features that produce the most accurate results. This is achieved by setting some features to “1” (selected) and others to “0” (not selected), as shown in Fig. 2. At the start; each solution is represented by binary values of either “0” or “1”.

**Evaluation** The FS has multiple objectives since it must achieve two goals: reduce the count of features that must be

selected and increase the classifier’s accuracy. To achieve this balance, the fitness function is designed to evaluate the solutions and meet both objectives simultaneously in Eq. (7), where  $\gamma_R(D)$  represents the rate of classification error of R, the condition attribute set in respect to decision D, which is computed via the ML classifier KNN.

$$fitness = \alpha \gamma_R(D) + \beta \frac{|s|}{|d|} \quad (7)$$

The span of the chosen attribute subset’s cardinality is represented by the symbol  $|s|$ . Each sample of the actual dataset that contains features is represented by a cardinality of  $|d|$ .  $\alpha$  and  $\beta$  are the two constraints of weight, which are equivalent to the value of the classification\_accuracy and

Algorithm 1 The proposed 2M-GWO

---

**Inputs:**  $n$ , Number of grey wolves in the population.  
 $iterations$ , maximum number of iterations.

1. Initialize a population of  $n$  grey wolves  $X_i$ ,  $i = 1$  to  $n$
2. Initialize parameters  $a$ ,  $A$ , and  $C$
3. Calculate the fitness of each grey wolf
4. Assign the best three grey wolves to  $X_\alpha$ ,  $X_\beta$ , and  $X_\delta$  respectively
5. Set  $t = 0$
6. while  $t < iterations$  do
  - for each grey wolf  $X_i$  do
    - Update the position of  $X_i$  using Eq. (6)
  - end for
    - Update parameters  $a$ ,  $A$ , and  $C$
    - Convert the position of grey wolves to binary using Eq. (8) and Eq. (9)
    - Calculate the fitness of all grey wolves
    - Update the best three grey wolves  $X_\alpha$ ,  $X_\beta$ , and  $X_\delta$
    - Apply two-phase mutation to  $X_\alpha$  and update its fitness
    - Increment  $t$  by 1
- end while

return the best grey wolf  $X_\alpha$

---

**Output:** The best grey wolf  $X_\alpha$

---

the length of the subset of the chosen feature,  $\alpha \in [0,1]$  and  $\alpha + \beta = 1$ . The evaluation function will overlook results that could have the equivalent accuracy but use fewer attributes, which is a key component in reducing the dimensionality problem if it solely considers classification accuracy.

### Transformation function

The GW's positions produced by the typical GWO are continuous in nature. Since it conflicts with the binary structure of the FS, it cannot be as it is applied to our case. Since the issue with FS arises from picking (1) or ignoring (0) the advantageous attributes that improve the capability of the classifier. The task of changing the continuous value of the search space to binary values is performed by the transformation function (TF). The TF used in this study is an S-shaped sigmoidal function [15].

Conversion of binary values to continuous values is done by using the following Eqs. (8) and (9) of the sigmoidal transformation function:

$$x_i = \frac{1}{1 + e^{-x_i}} \quad (8)$$

$$x_{binary} = \begin{cases} 0 & \text{if } rand < x_i \\ 1 & \text{if } rand \geq x_i \end{cases} \quad (9)$$

where  $x_i$  depicts the continuous value of the feature, the value of  $i$  varies from 1 to  $d$ ,  $x_{binary}$  can have a value of 0 (not selected) or 1 (selected) on comparing the  $x_i$  with a random value  $rand$ .

### Two-phase mutation

A mutation operator is utilized to enhance the performance of the proposed algorithm. In the initial phase of mutation, the primary objective is to reduce the number of selected attributes while preserving high classification accuracy [13]. The subsequent phase focuses on improving classification efficacy by introducing more nuanced attributes. To mitigate delays that may arise from the mutation operator, step 4 can be executed with a mutation probability ( $Mp$ ). This approach optimizes the mutation process, allowing for controlled adjustments without compromising efficiency.

Algorithm 1 provides the detailed pseudocode for the proposed mutation process. Additionally, the complete pseudocode for the 2M-GWO algorithm, incorporating both mutation and other key components, is outlined in Algorithm 2.

In Algorithm 2,  $X_\alpha$ ,  $X_\beta$ , and  $X_\delta$  represent the positions of the best three grey wolves (alpha, beta, delta), which

## Algorithm 2 Two-Phase Mutation

---

**Input:**  $X\alpha$ : the best grey wolf from each iteration

1. Calculate the fitness of  $X\alpha$  and store it as *fitness*
2. Initialize *one\_positions* to store the indices of selected features in  $X\alpha$
3. Set  $X_{mutated1} = X\alpha$

// First Mutation Phase

4. for each index  $i$  in *one\_positions* do
  - Generate a random number  $r$
  - if  $r < Mp$  then
    - Set  $X_{mutated1}[i] = 0$
    - Calculate the fitness of  $X_{mutated1}$  and store it as *fitness\_mutated*
    - if *fitness\_mutated* < *fitness* then
      - Set *fitness* = *fitness\_mutated*
      - Set  $X\alpha = X_{mutated1}$
  - end if
- end for

// Second Mutation Phase

5. Initialize *zero\_positions* to store the indices of unselected features in  $X\alpha$
6. Set  $X_{mutated2} = X\alpha$
7. for each index  $j$  in *zero\_positions* do
  - Generate a random number  $r$
  - if  $r < Mp$  then
    - Set  $X_{mutated2}[j] = 1$
    - Calculate the fitness of  $X_{mutated2}$  and store it as *fitness\_mutated*
    - if *fitness\_mutated* < *fitness* then
      - Set *fitness* = *fitness\_mutated*
      - Set  $X\alpha = X_{mutated2}$
  - end if
- end for

return  $X\alpha$

**Output:** The improved  $X\alpha$

---

guide the search process.  $X_i$  denotes the position of the  $i^{\text{th}}$  grey wolf. *Fitness* is the variable which shows the fitness value of  $X\alpha$ . *one\_positions* is a vector storing the positions of selected features in  $X\alpha$ , and *zero\_positions* store the positions of unselected features in  $X\alpha$ .

**Computational complexity of the 2M-GWO technique** The overall computational complexity of the algorithm primarily pivots on the number of iterations  $t$ , the number of Grey Wolves  $n$ , the dimensionality  $d$  of each wolf, and the complexity  $f$  of the fitness evaluation function. The computational complexity expression,  $O(t \times n \times (d + f))$ , suggests that the algorithm scales linearly with the number of iterations and the number of wolves but also depends linearly on the sum of the dimensionality and

the complexity of the fitness function. This indicates that while the algorithm can handle moderate sizes efficiently, large values or complex fitness functions could significantly increase computational demands.

### 3.3 Classifiers

The literature has a number of learning classifiers. As a result, we restrict our research to using only five distinct classifiers:

### 3.3.1 Support vector machine (SVM)

It aims to find the best hyperplane in a high-dimensional space that can successfully separate different classes. The hyperplane is defined by a set of parameters that are optimized during training to maximize the margin between the hyperplane and the closest data points from each class. One of the main advantages of SVM is its ability to handle high-dimensional data and small sample sizes [16].

### 3.3.2 Random forest (RF)

This is an example of an ensemble learning method used for classification, which involves creating multiple Decision Trees at training time and then combining the outputs of these trees to make a final decision. RF uses a bagging technique to train each DT on a random subset of the data used for training, which reduces overfitting and improves the generalizability of a model [17].

### 3.3.3 Gradient boosting (GB)

This is an ML technique that is commonly used for classifying software into defective and non-defective modules. It involves accumulating weak decision trees to a model on a recurring basis, with each new tree fitting the residual errors of the preceding tree. The model then makes a final decision by merging the outputs of all of the trees in the model. Gradient boosting can be used with a variety of loss functions, allowing it to handle a range of classification problems [18].

### 3.3.4 AdaBoost (AB)

The AB is also known as adaptive boosting, which trains a sequence of weak classifiers on the same dataset. Each subsequent classifier is trained to give more weight to the samples that were misclassified by the previous classifiers. Finally, the algorithm combines the predictions of all the weak classifiers to make a final prediction [10].

### 3.3.5 K-nearest neighbour (KNN)

The KNN works on the distance-based phenomenon. It is a non-parametric algorithm and is not dependent on the distribution of the given dataset. The KNN algorithm works by first training on a labelled dataset, and then using the learned information to classify new, unlabelled data points based on their resemblance to the training examples. The KNN algorithm determines the similarity between data points by calculating the distance among them, often using Euclidean distance [19].

## 3.4 Evaluation method

To assess the model's performance and facilitate result comparison, we employ ROC\_AUC. It is a graphical tool that assesses a binary classifier's effectiveness by varying the discrimination threshold value. Ranging from 0 to 1, a score of 1 indicates a flawless classifier, while 0.5 signifies performance no better than random chance. The ROC\_AUC curve illustrates the True Positive Rate (TPR) against the False Positive Rate (FPR) across different threshold values [20]. The True Positive Rate (TPR) is computed by dividing the number of correctly identified positive instances by the total number of positive instances, whereas the False Positive Rate (FPR) is determined by dividing the number of incorrectly identified positive instances by the total number of negative instances [21].

A high ROC\_AUC value indicates that the classifier is able to distinguish between positive and negative cases well, while a low ROC\_AUC value indicates poor performance. ROC\_AUC is a popular evaluation metric other than accuracy in machine learning because it is insensitive to imbalances in class distribution and is well-suited for imbalanced datasets. Additionally, the ROC\_AUC can be used to compare the performance of different classifiers.

## 3.5 Parameter setting

To address the stochastic character, each algorithm is performed 20 times independently using a random seed. The maximum number of iterations for all subsequent tests is set at 100. There are 5 SA in the entire population. Additionally, 10-CV is used in this case. In order to choose the optimal values for  $\alpha$  and  $\beta$  and based on certain values collected from past studies, several tests are carried out on various datasets. As a consequence,  $\alpha$  and  $\beta$  are set to corresponding values of 0.01 and 0.99. To enable a fair comparison of the algorithms, the parameter specifications of the techniques are collected from the existing studies. The parameters of some of the optimization techniques are mentioned in Table 3. For all other techniques, the default value is taken.

## 3.6 Statistical test

In software defect prediction, the Friedman test is often employed to evaluate and compare the effectiveness of various defect prediction models or techniques. This is crucial because software defect prediction models are designed to predict the likelihood of defects in software modules, and choosing the most effective model can significantly impact the quality and reliability of software.

**Table 3** Parameter settings for optimization techniques

| Technique | Parameter                | Value |
|-----------|--------------------------|-------|
| 2M-GWO    | Mutation probability, Mp | 0.5   |
| SCO       | Constant value, a        | 2     |
| WO        | Spiral coefficient, b    | 1     |

The Friedman test serves as a non-parametric statistical method utilized for identifying variations among treatments across numerous test iterations. In the context of software defect prediction [22], it is particularly useful because it allows researchers to compare the performance of different prediction models or methods over multiple datasets without assuming a normal distribution of the data.

## 4 Result analysis

This subsection provides the findings in response to the Research Questions (RQ) as follows:

**RQ 1** What is the effect of FS techniques on the SDP models? Does 2M-GWO significantly improve the performance of SDP models as compared to GWO and WFS Models?

Tables 4, 5, 6, 7, 8 and 9 explores the experimental results of the original GWO, modified 2M-GWO and WFS model for 27 datasets using five classifiers, namely SVM, RF, GB, AB and KNN. The experimental data include the feature size and ROC\_AUC value for each dataset. We employed SMOTE to balance the classes in the given dataset. The ROC\_AUC value for 27 open-source datasets is obtained by taking the average value of 20 runs for each project.

The data presented in Table 4 clearly demonstrates that the 2M-GWO algorithm outperforms the other two algorithms, showing promising results in terms of ROC\_AUC values across a majority of projects. Analysis of average ROC\_AUC values indicates that the modified 2M-GWO algorithm surpasses the original GWO and WFS methods, with a maximum average value of 0.785. Furthermore, the proposed algorithm, in conjunction with SVM, significantly improved average ROC\_AUC values by over 6% across all datasets, demonstrating strong overall performance.

Table 5 illustrates the results of the RF classifier utilized to evaluate the original GWO, the modified 2M-GWO, and the complete dataset. It is evident that the 2M-GWO algorithm outperforms the other two methods, as it demonstrates the highest ROC\_AUC values across a larger number of projects. The enhanced 2M-GWO surpasses the original GWO and WFS techniques in terms of average

**Table 4** ROC\_AUC by SVM

| Projects           | GWO          | 2M-GWO       | WFS          |
|--------------------|--------------|--------------|--------------|
| PROMISE-Ant-1.7    | 0.821        | 0.815        | <b>0.88</b>  |
| PROMISE-Camel-1.2  | 0.515        | <b>0.618</b> | <b>0.618</b> |
| PROMISE-Camel-1.4  | <b>0.704</b> | 0.629        | 0.629        |
| PROMISE-Jedit-3.2  | 0.779        | <b>0.81</b>  | 0.782        |
| PROMISE-Jedit-4.1  | <b>0.87</b>  | 0.783        | 0.725        |
| PROMISE-Jedit-4.2  | 0.747        | <b>0.83</b>  | 0.725        |
| PROMISE-Log4j-1.0  | 0.912        | <b>0.914</b> | 0.9          |
| PROMISE-Log4j-1.1  | 0.867        | 0.88         | <b>1</b>     |
| PROMISE-Lucene-2.0 | 0.765        | <b>0.807</b> | <b>0.807</b> |
| PROMISE-Lucene-2.2 | <b>0.656</b> | 0.589        | 0.589        |
| PROMISE-Tomcat     | 0.709        | <b>0.737</b> | <b>0.737</b> |
| PROMISE-Xalan 2.4  | <b>0.757</b> | 0.672        | 0.672        |
| PROMISE-Xalan 2.5  | 0.603        | <b>0.752</b> | 0.647        |
| PROMISE-Xalan-2.6  | 0.649        | <b>0.961</b> | 0.752        |
| AEEEM-EQ           | 0.801        | 0.803        | <b>0.825</b> |
| AEEEM-JDT          | 0.836        | <b>0.848</b> | 0.556        |
| AEEEM-LC           | 0.801        | <b>0.868</b> | 0.556        |
| AEEEM-ML           | 0.772        | <b>0.798</b> | 0.556        |
| AEEEM-PDE          | 0.653        | <b>0.755</b> | 0.667        |
| NASA-Cm1           | 0.751        | <b>0.795</b> | 0.698        |
| NASA-Mw1           | 0.578        | <b>0.777</b> | 0.683        |
| NASA-PC1           | 0.736        | 0.846        | <b>0.849</b> |
| NASA-PC3           | 0.812        | <b>0.896</b> | 0.852        |
| NASA-PC4           | 0.851        | 0.906        | <b>0.919</b> |
| RELINK-Apache      | 0.732        | <b>0.776</b> | 0.689        |
| RELINK-Safe        | 0.629        | 0.629        | <b>0.886</b> |
| RELINK-Zxing       | 0.667        | <b>0.714</b> | 0.705        |
| Avg                | 0.739        | 0.785        | 0.737        |

The significance of bold indicates the highest value for each dataset when performing the comparative analysis

ROC\_AUC values, with a maximum average value of 0.8. The average ROC\_AUC value for all datasets saw an increase of approximately 2% when utilizing the proposed algorithm.

The ROC\_AUC value calculated by Gradient Boosting (GB) for the full dataset and processed datasets, in which features were selected GWO and 2M-GWO, is presented in Table 6. The results indicate that the 2M-GWO-based model demonstrates superior performance compared to the GWO and WFS cases, with an average improvement of 2.3% in the ROC\_AUC value. In approximately 40% of datasets, the 2M-GWO-based model outperformed the others, while in approximately 19% of datasets, its performance was comparable to either the GWO-based model or the full dataset.

Table 7 presents the computed ROC\_AUC value by AB for the entire datasets and the processed datasets, where the

**Table 5** ROC\_AUC by RF

| Projects           | GWO          | 2M-GWO       | WFS          |
|--------------------|--------------|--------------|--------------|
| PROMISE-Ant-1.7    | 0.832        | <b>0.845</b> | 0.843        |
| PROMISE-Camel-1.2  | 0.641        | 0.634        | <b>0.652</b> |
| PROMISE-Camel-1.4  | <b>0.736</b> | 0.71         | 0.715        |
| PROMISE-Jedit-3.2  | 0.775        | <b>0.776</b> | 0.758        |
| PROMISE-Jedit-4.1  | <b>0.886</b> | 0.862        | 0.692        |
| PROMISE-Jedit-4.2  | <b>0.733</b> | 0.73         | 0.692        |
| PROMISE-Log4j-1.0  | 0.879        | <b>0.889</b> | 0.871        |
| PROMISE-Log4j-1.1  | 0.876        | <b>1</b>     | <b>1</b>     |
| PROMISE-Lucene-2.0 | 0.73         | <b>0.767</b> | 0.749        |
| PROMISE-Lucene-2.2 | 0.563        | <b>0.616</b> | <b>0.616</b> |
| PROMISE-Tomcat     | 0.811        | <b>0.83</b>  | 0.824        |
| PROMISE-Xalan 2.4  | <b>0.754</b> | 0.749        | 0.715        |
| PROMISE-Xalan 2.5  | 0.656        | <b>0.827</b> | 0.748        |
| PROMISE-Xalan- 2.6 | 0.82         | <b>1</b>     | 0.814        |
| AEEEM-EQ           | 0.818        | <b>0.841</b> | 0.825        |
| AEEEM-JDT          | 0.877        | <b>0.881</b> | 0.85         |
| AEEEM-LC           | <b>0.884</b> | 0.821        | 0.849        |
| AEEEM-ML           | <b>0.859</b> | 0.838        | 0.857        |
| AEEEM-PDE          | 0.787        | 0.784        | <b>0.793</b> |
| NASA-Cm1           | 0.623        | 0.718        | <b>0.766</b> |
| NASA-Mw1           | <b>0.733</b> | 0.5          | 0.613        |
| NASA-PC1           | 0.927        | 0.925        | <b>0.93</b>  |
| NASA-PC3           | 0.825        | <b>0.835</b> | 0.827        |
| NASA-PC4           | 0.919        | 0.931        | <b>0.938</b> |
| RELINK-Apache      | <b>0.732</b> | 0.697        | 0.68         |
| RELINK-Safe        | 0.786        | 0.786        | <b>0.971</b> |
| RELINK-Zxing       | 0.611        | <b>0.827</b> | 0.807        |
| Avg                | 0.780        | <b>0.800</b> | 0.792        |

The significance of bold indicates the highest value for each dataset when performing the comparative analysis

features were selected by GWO and 2M-GWO. It is observed that the 2M-GWO-based model outperforms the GWO and WFS case in terms of overall performance, with an average ROC\_AUC value improvement of 4.6%. Specifically, for approximately 33% of the datasets, the 2M-GWO-based model performs better, while for around 11% of the datasets, it performs similarly to either of the two approaches.

According to the data presented in Table 8, the 2M-GWO-KNN demonstrates superior classification performance compared to the other model in over 50% of the datasets, as indicated by the ROC\_AUC value. The average ROC\_AUC value for the overall performance of 2M-GWO shows an increase of approximately 4% when compared to the other two models.

Based on the analysis presented in Table 9 and Fig. 3, on taking an average value, the 2M-GWO algorithm achieved an approximate 80% reduction in feature sets

**Table 6** ROC\_AUC by GB

| Projects           | GWO          | 2M-GWO       | WFS          |
|--------------------|--------------|--------------|--------------|
| PROMISE-Ant-1.7    | <b>0.78</b>  | 0.71         | 0.71         |
| PROMISE-Camel-1.2  | 0.514        | <b>0.551</b> | <b>0.551</b> |
| PROMISE-Camel-1.4  | 0.726        | <b>0.673</b> | <b>0.673</b> |
| PROMISE-Jedit-3.2  | <b>0.823</b> | 0.771        | 0.771        |
| PROMISE-Jedit-4.1  | 0.786        | <b>0.819</b> | 0.702        |
| PROMISE-Jedit-4.2  | 0.687        | <b>0.731</b> | 0.702        |
| PROMISE-Log4j-1.0  | 0.7          | <b>0.8</b>   | 0.764        |
| PROMISE-Log4j-1.1  | 0.752        | <b>0.933</b> | 0.933        |
| PROMISE-Lucene-2.0 | <b>0.754</b> | 0.728        | 0.728        |
| PROMISE-Lucene-2.2 | <b>0.714</b> | <b>0.714</b> | <b>0.714</b> |
| PROMISE-Tomcat     | 0.758        | <b>0.805</b> | <b>0.805</b> |
| PROMISE-Xalan 2.4  | <b>0.731</b> | 0.72         | 0.72         |
| PROMISE-Xalan 2.5  | 0.663        | 0.656        | <b>0.666</b> |
| PROMISE-Xalan- 2.6 | 0.798        | <b>0.983</b> | 0.797        |
| AEEEM-EQ           | 0.836        | <b>0.841</b> | 0.828        |
| AEEEM-JDT          | 0.78         | <b>0.82</b>  | 0.784        |
| AEEEM-LC           | <b>0.868</b> | 0.803        | 0.764        |
| AEEEM-ML           | 0.787        | <b>0.805</b> | 0.804        |
| AEEEM-PDE          | 0.737        | 0.697        | <b>0.748</b> |
| NASA-Cm1           | 0.519        | <b>0.656</b> | 0.588        |
| NASA-Mw1           | <b>0.613</b> | 0.561        | 0.535        |
| NASA-PC1           | 0.9          | 0.895        | <b>0.907</b> |
| NASA-PC3           | <b>0.81</b>  | 0.798        | 0.801        |
| NASA-PC4           | 0.911        | 0.926        | <b>0.931</b> |
| RELINK-Apache      | 0.683        | <b>0.789</b> | 0.684        |
| RELINK-Safe        | <b>0.857</b> | <b>0.857</b> | 0.7          |
| RELINK-Zxing       | 0.738        | <b>0.747</b> | 0.717        |
| Avg                | 0.749        | <b>0.767</b> | 0.741        |

The significance of bold indicates the highest value for each dataset when performing the comparative analysis

across various datasets. The NASA dataset showed the most significant reduction at around 86%, followed by the AEEEM and RELINK datasets, which saw an 80% reduction each. In contrast, the PROMISE dataset had the smallest reduction in features at 75%.

Table 10 presents the comparative performance of different feature selection techniques based on their mean rank achieved by Friedman's test, determined through the ROC\_AUC metric. The mean rank serves as a measure of each technique's efficacy in enhancing the prediction accuracy of a software defect prediction model. A lower mean rank indicates better performance. The p-value associated with these rankings is 0.022, suggesting that the differences in performance among these techniques are statistically significant at the 95% confidence interval. 2M-GWO stands out as the most effective technique, achieving the highest performance with a mean rank of 1.00, denoted as rank I. GWO is ranked second with a mean rank of 2.40,



**Table 7** ROC\_AUC by AB

| Projects           | GWO          | 2M-GWO       | WFS          |
|--------------------|--------------|--------------|--------------|
| PROMISE-Ant-1.7    | 0.741        | <b>0.793</b> | 0.693        |
| PROMISE-Camel-1.2  | 0.507        | <b>0.517</b> | 0.517        |
| PROMISE-Camel-1.4  | <b>0.732</b> | 0.714        | 0.714        |
| PROMISE-Jedit-3.2  | <b>0.791</b> | 0.78         | 0.78         |
| PROMISE-Jedit-4.1  | 0.789        | <b>0.822</b> | 0.714        |
| PROMISE-Jedit-4.2  | 0.673        | 0.692        | <b>0.714</b> |
| PROMISE-Log4j-1.0  | 0.757        | <b>0.793</b> | 0.657        |
| PROMISE-Log4j-1.1  | 0.752        | <b>0.933</b> | <b>0.933</b> |
| PROMISE-Lucene-2.0 | <b>0.77</b>  | 0.767        | 0.767        |
| PROMISE-Lucene-2.2 | 0.573        | <b>0.704</b> | <b>0.704</b> |
| PROMISE-Tomcat     | 0.659        | <b>0.731</b> | 0.721        |
| PROMISE-Xalan 2.4  | <b>0.738</b> | 0.734        | 0.724        |
| PROMISE-Xalan 2.5  | 0.68         | <b>0.806</b> | 0.648        |
| PROMISE-Xalan- 2.6 | 0.807        | <b>1</b>     | 0.806        |
| AEEEM-EQ           | 0.766        | <b>0.845</b> | 0.818        |
| AEEEM-JDT          | 0.819        | 0.825        | <b>0.831</b> |
| AEEEM-LC           | <b>0.845</b> | 0.778        | 0.577        |
| AEEEM-ML           | 0.787        | 0.786        | <b>0.803</b> |
| AEEEM-PDE          | 0.673        | 0.643        | <b>0.702</b> |
| NASA-Cm1           | 0.588        | 0.603        | <b>0.642</b> |
| NASA-Mw1           | 0.57         | 0.6          | <b>0.613</b> |
| NASA-PC1           | 0.909        | 0.899        | <b>0.916</b> |
| NASA-PC3           | <b>0.796</b> | 0.768        | 0.705        |
| NASA-PC4           | 0.91         | 0.92         | <b>0.924</b> |
| RELINK-Apache      | 0.732        | <b>0.775</b> | 0.589        |
| RELINK-Safe        | <b>0.914</b> | <b>0.914</b> | 0.729        |
| RELINK-Zxing       | 0.601        | 0.715        | <b>0.719</b> |
| Avg                | 0.736        | <b>0.772</b> | 0.728        |

The significance of bold indicates the highest value for each dataset when performing the comparative analysis

labelled as rank II. This shows that the standard Grey Wolf Optimiser, while effective, is outperformed by its two-phase variant in the context of feature selection for defect prediction. WFS is closely ranked third with a mean rank of 2.60, marked as rank III.

**Summary:** RQ1 reveals that it is apparent that the 2M-GWO method is an effective approach for reducing feature size while maintaining model performance for SDP for all five datasets. The 2M-GWO technique emerges as the most effective approach, achieving the highest level of performance with a mean rank of 1.00. Additionally, the 2M-GWO algorithm facilitated an average reduction of approximately 80% in feature sets across multiple datasets

**Table 8** ROC\_AUC value calculated by KNN

| Projects           | GWO          | 2M-GWO       | WFS          |
|--------------------|--------------|--------------|--------------|
| PROMISE-Ant-1.7    | 0.741        | 0.806        | <b>0.815</b> |
| PROMISE-Camel-1.2  | 0.669        | <b>0.693</b> | 0.643        |
| PROMISE-Camel-1.4  | <b>0.699</b> | 0.686        | 0.686        |
| PROMISE-Jedit-3.2  | 0.733        | <b>0.84</b>  | <b>0.84</b>  |
| PROMISE-Jedit-4.1  | <b>0.874</b> | 0.855        | 0.638        |
| PROMISE-Jedit-4.2  | <b>0.793</b> | 0.654        | 0.638        |
| PROMISE-Log4j-1.0  | 0.821        | <b>0.868</b> | 0.829        |
| PROMISE-Log4j-1.1  | 0.943        | <b>0.948</b> | <b>0.948</b> |
| PROMISE-Lucene-2.0 | 0.783        | 0.788        | <b>0.79</b>  |
| PROMISE-Lucene-2.2 | <b>0.643</b> | 0.624        | 0.584        |
| PROMISE-Tomcat     | 0.628        | <b>0.647</b> | <b>0.647</b> |
| PROMISE-Xalan 2.4  | 0.719        | <b>0.79</b>  | <b>0.79</b>  |
| PROMISE-Xalan 2.5  | 0.629        | <b>0.81</b>  | 0.665        |
| PROMISE-Xalan- 2.6 | 0.814        | <b>1</b>     | 0.81         |
| AEEEM-EQ           | 0.782        | 0.84         | <b>0.871</b> |
| AEEEM-JDT          | <b>0.842</b> | 0.78         | 0.763        |
| AEEEM-LC           | 0.848        | <b>0.851</b> | 0.763        |
| AEEEM-ML           | 0.785        | 0.786        | <b>0.811</b> |
| AEEEM-PDE          | 0.657        | <b>0.783</b> | 0.715        |
| NASA-Cm1           | 0.651        | 0.706        | <b>0.759</b> |
| NASA-Mw1           | 0.596        | 0.524        | <b>0.624</b> |
| NASA-PC1           | 0.905        | <b>0.941</b> | 0.852        |
| NASA-PC3           | 0.78         | 0.791        | <b>0.833</b> |
| NASA-PC4           | 0.852        | <b>0.905</b> | 0.827        |
| RELINK-Apache      | 0.683        | <b>0.689</b> | 0.671        |
| RELINK-Safe        | 0.814        | <b>0.814</b> | 0.786        |
| RELINK-Zxing       | 0.67         | 0.782        | <b>0.793</b> |
| Avg                | 0.753        | <b>0.785</b> | 0.755        |

The significance of bold indicates the highest value for each dataset when performing the comparative analysis

**RQ 2** Which is the best-performing ML technique for predicting software defects?

Table 11 displays the mean ranks obtained by the Friedman's test on the basis of ROC\_AUC for the ML techniques used in the study. 2M-GWO-RF achieved the best mean rank of 2.19, indicating its superior performance in terms of prediction accuracy and efficiency. 2M-GWO-KNN attained a mean rank of 2.94, placing it second in terms of performance. While it is not as effective as 2M-GWO-RF, it still demonstrates good performance and is viable with a mean rank of 3.15; SVM, along with 2MGWO, is ranked third in terms of performance. AB and GB are the ensemble classifiers with a mean rank of 3.35 and 3.37, ranking fourth and last among the classifiers evaluated. The p-value of 0.034 indicates that the differences in the mean ranks of these techniques are statistically significant. Overall, the combination of 2M-GWO and

**Table 9** Number of features calculated by different techniques

| Projects           | GWO       | 2M-GWO    | WFS |
|--------------------|-----------|-----------|-----|
| PROMISE-Ant-1.7    | <b>5</b>  | 6         | 20  |
| PROMISE-Camel-1.2  | <b>6</b>  | <b>6</b>  | 20  |
| PROMISE-Camel-1.4  | <b>5</b>  | 6         | 20  |
| PROMISE-Jedit-3.2  | <b>5</b>  | <b>5</b>  | 20  |
| PROMISE-Jedit-4.1  | 5         | <b>4</b>  | 20  |
| PROMISE-Jedit-4.2  | <b>4</b>  | <b>4</b>  | 20  |
| PROMISE-Log4j-1.0  | <b>4</b>  | <b>4</b>  | 20  |
| PROMISE-Log4j-1.1  | <b>3</b>  | <b>3</b>  | 20  |
| PROMISE-Lucene-2.0 | 6         | <b>5</b>  | 20  |
| PROMISE-Lucene-2.2 | 7         | <b>6</b>  | 20  |
| PROMISE-Tomcat     | <b>4</b>  | <b>4</b>  | 20  |
| PROMISE-Xalan 2.4  | 6         | <b>5</b>  | 20  |
| PROMISE-Xalan 2.5  | 10        | <b>7</b>  | 20  |
| PROMISE-Xalan- 2.6 | 7         | <b>6</b>  | 20  |
| AEEEM-EQ           | <b>11</b> | 12        | 61  |
| AEEEM-JDT          | 14        | <b>13</b> | 61  |
| AEEEM-LC           | <b>6</b>  | <b>6</b>  | 61  |
| AEEEM-ML           | <b>12</b> | 14        | 61  |
| AEEEM-PDE          | <b>12</b> | 13        | 61  |
| NASA-Cm1           | 4         | <b>2</b>  | 37  |
| NASA-Mw1           | <b>4</b>  | <b>4</b>  | 37  |
| NASA-PC1           | <b>4</b>  | <b>4</b>  | 37  |
| NASA-PC3           | 8         | <b>6</b>  | 37  |
| NASA-PC4           | <b>8</b>  | <b>8</b>  | 37  |
| RELINK-Apache      | <b>5</b>  | 6         | 26  |
| RELINK-Safe        | 3         | 3         | 26  |
| RELINK-Zxing       | 8         | <b>7</b>  | 26  |
| Avg                | 7         | <b>6</b>  | 31  |

The significance of bold indicates the highest value for each dataset when performing the comparative analysis

Random Forest (2M-GWO-RF) appears to offer the best performance among the tested classification techniques.

**Summary:** From RQ 2, it is evident that the combination of 2M-GWO and Random Forest (2M-GWO-RF) is the best-performing machine-learning technique for predicting software defects, as indicated by its superior mean rank of 2.19

### RQ 3 Which software metrics are frequently selected using the proposed algorithm?

The analysis of feature selection across various datasets using the 2M-GWO FS technique is detailed in Figs. 4, 5, 6 and 7. These figures graphically represent the selection frequency of specific software metrics (features) crucial for predicting software defects. The x-axis of each figure indicates different software metrics derived from the

**Table 10** Mean Rank obtained on the basis of ROC\_AUC for feature selection technique

| Techniques | Mean rank (p value = 0.022) |
|------------|-----------------------------|
| 2M-GWO     | 1.00 (I)                    |
| GWO        | 2.40 (II)                   |
| WFS        | 2.60 (III)                  |

**Table 11** Mean Rank obtained on the basis of ROC\_AUC for ML techniques

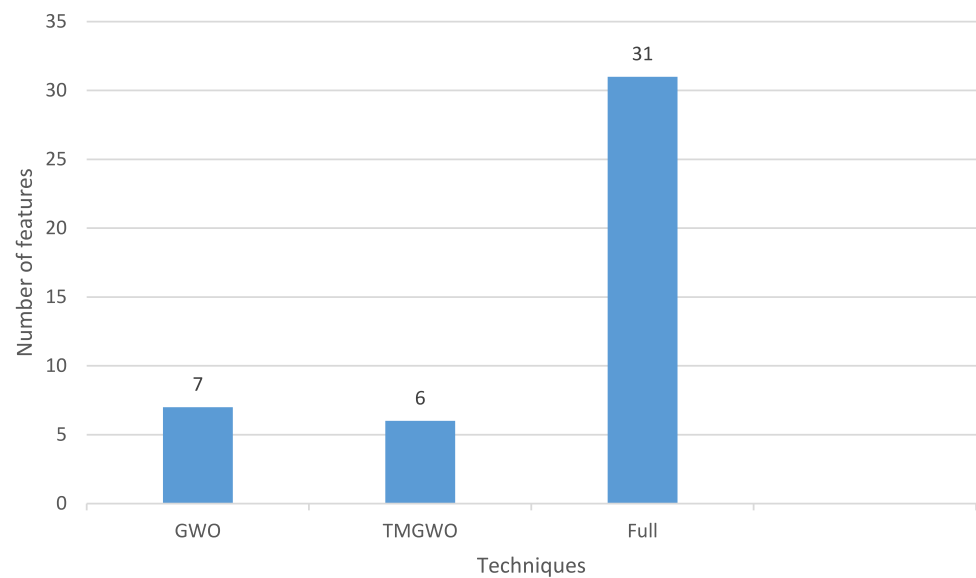
| Classification Techniques | Mean rank (p value = .034) |
|---------------------------|----------------------------|
| 2M-GWO-RF                 | 2.19 (I)                   |
| 2M-GWO-KNN                | 2.94 (II)                  |
| 2M-GWO-SVM                | 3.15 (III)                 |
| 2M-GWO-AB                 | 3.35 (IV)                  |
| 2M-GWO-GB                 | 3.37 (V)                   |

datasets, while the y-axis quantifies how frequently each metric was selected across multiple iterations.

For the RELINK dataset (Fig. 4), metrics such as CountLineCodeDecl, CountLineBlank, and CountLineCode show high selection frequencies of 26, 21, and 20 times, respectively, out of 60 total runs distributed across three projects (each executed 20 times). In the case of the PROMISE dataset (Fig. 5), the object-oriented metrics rfc, ce, and npm were selected 131 and 102 times each out of 280 iterations. Regarding the AEEEM dataset (Fig. 6), which includes 61 process metrics across five projects, metrics such as CVSEntropy, numberOfNontrivialBugsFoundUntil, and Ck OO numberOfMethodsInherited were most frequently selected, with counts of 49, 45, and 41 respectively, out of 100 runs. For the NASA project (Fig. 7), which focuses on Halstead's and size metrics, the metrics LOC CODE AND COMMENT, LOC BLANK, and LOC EXECUTABLE were identified as the most frequently occurring, with each being selected 28, 25, and 25 times respectively in twenty runs across five projects.

**Summary:** Analysis from RQ3 reveals key patterns in metric selection across different datasets. For the RELINK dataset, metrics such as CountLineCodeDecl, CountLineBlank, and CountLineCode were consistently chosen. In the PROMISE dataset, the most frequently selected metrics were object-oriented measures, including rfc, ce, and npm. In the AEEEM dataset, encompassing 61 process metrics across five projects, CVSEntropy, numberOfNontrivialBugsFoundUntil, and Ck OO numberOfMethodsInherited were the metrics most often chosen. In the NASA project, which focuses on Halstead's size metrics, LOC CODE AND COMMENT, LOC BLANK, and LOC EXECUTABLE were the most prominent metrics. The frequent selection of these metrics underscores their importance in the defect prediction process, suggesting that they provide valuable insights into software quality and should be considered in data collection for future projects

**Fig. 3** Comparison of feature selection techniques on the basis of the number of features selected



**RQ 4** What is the predictive performance of the proposed model as compared to other benchmark FS models for SDP?

In order to conduct a comprehensive comparative analysis, two types of analysis are made.

In first type, the proposed algorithm is evaluated against five closely related meta-heuristic algorithms: Harris Hawks Optimization (HHO) [23], Salp Swarm Optimization (SSO) [24], Whale Optimization (WO) [25], Jaya Optimization (JO) [26], and Sine Cosine Optimization (SCO) [27]. These comparisons, summarized in Table 12, were carried out under consistent experimental conditions to ensure fairness and accuracy in the results. Each algorithm was implemented and executed within the same computational environment, adhering to identical parameter settings. This approach facilitated direct comparisons and minimized variations that could arise from differing configurations. The ROC\_AUC measure was used as the primary basis for evaluating performance across all algorithms, as it provides a comprehensive measure of an algorithm's ability to distinguish between classes or optimize solutions effectively. By examining the AUC scores, we analyzed the fact that the proposed technique outperforms the other techniques in approximately 55.5% of the total datasets. SSO obtained a promising outcome in approximately 18.5% of the datasets. HHO and WO resulted in the best ROC\_AUC value for two datasets,

whereas SCO and JO outperformed for only one dataset each. This showcases the effectiveness of the proposed method in selecting the most pertinent features, which notably boost the performance of the SDP model.

In the second type, the performance outcomes of established approaches are sourced from existing literature and subjected to a thorough comparison. In the RKEE-based model, Riaz et al. [28] proposed two-stage data pre-processing which incorporates FS and a new rough set Easy Ensemble scheme, which shows an average value of 0.771, whereas the existing model proposed by Tumar et al. [29] was based on improved binary Moth Flame Optimization with Adaptive synthetic sampling (ADASYN) to predict software defects. BMFO is employed as a wrapper FS, while ADASYN enhances the input dataset, addresses the imbalanced dataset and shows a ROC\_AUC value of 0.7552. When compared with the existing methods based on evolutionary FS techniques, the proposed model based on 2M-GWO and Random Forest classifier obtained a ROC\_AUC of 0.800, which is better than the existing methods.

---

**Summary:** Research Question 4 demonstrates that the proposed model, integrating 2M-GWO with Random Forest, surpasses the performance of five related metaheuristic techniques in more than 55% of the datasets. Furthermore, it exhibits competitive results when evaluated against current state-of-the-art approaches

---

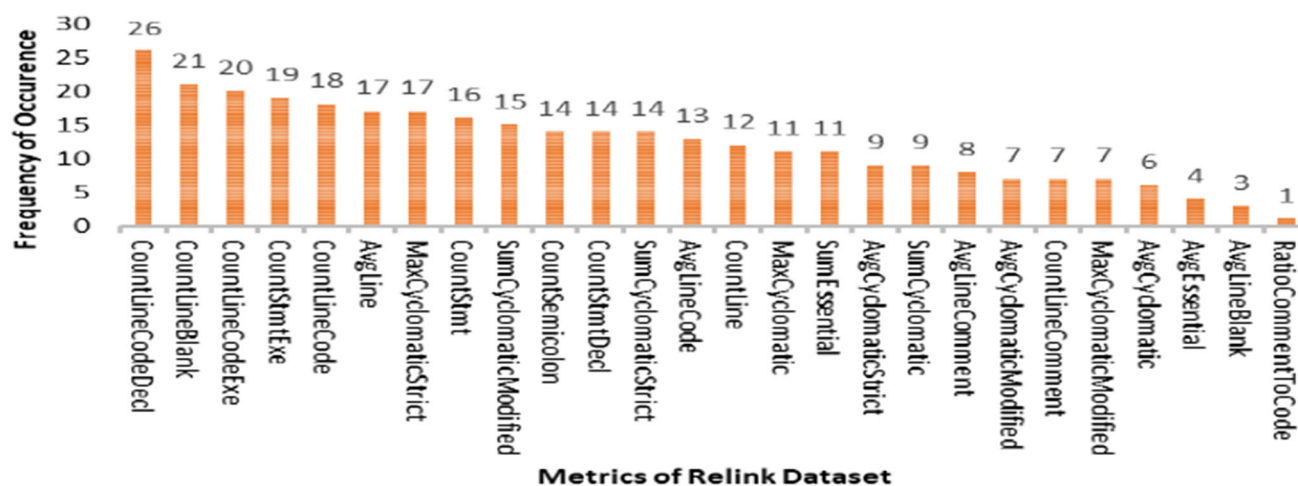


Fig. 4 Occurrence of metrics for RELINK dataset

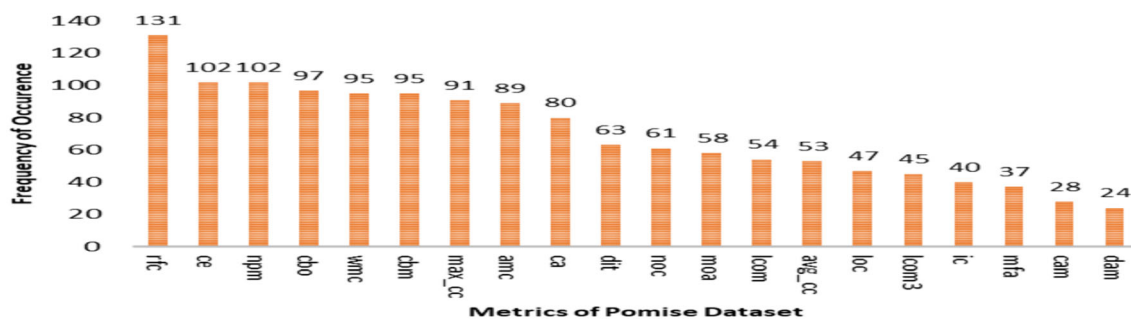


Fig. 5 Occurrence of metrics for PROMISE dataset

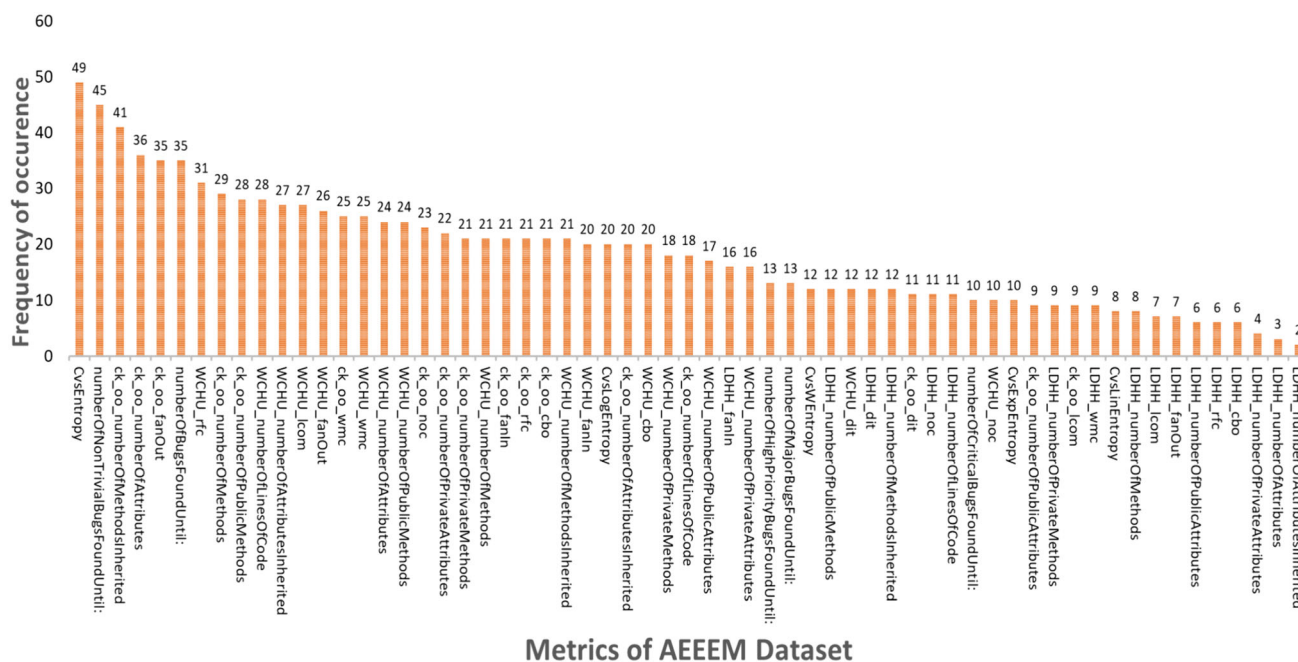


Fig. 6 Occurrence of metrics for AEEEM dataset

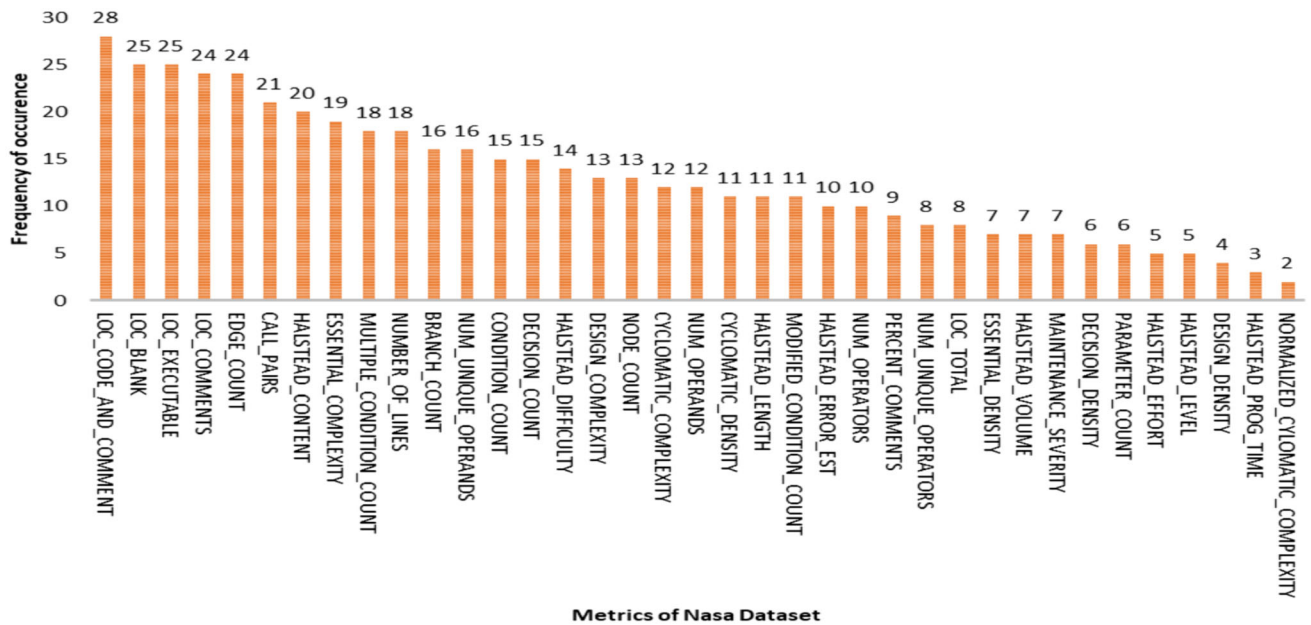


Fig. 7 Occurrence of metrics for NASA dataset

**Table 12** ROC\_AUC values of Proposed and other metaheuristic techniques

| Projects           | 2 M-GWO -RF  | SCO-RF       | SSO-RF       | JO-RF        | WO-RF        | HHO-RF       |
|--------------------|--------------|--------------|--------------|--------------|--------------|--------------|
| PROMISE-Ant-1.7    | <b>0.845</b> | 0.824        | 0.83         | 0.829        | 0.82         | 0.818        |
| PROMISE-Camel-1.2  | <b>0.634</b> | 0.625        | 0.604        | 0.615        | 0.63         | 0.615        |
| PROMISE-Camel-1.4  | 0.71         | 0.787        | 0.729        | 0.753        | 0.765        | <b>0.798</b> |
| PROMISE-Jedit-3.2  | 0.776        | 0.799        | 0.799        | 0.808        | 0.784        | <b>0.831</b> |
| PROMISE-Jedit-4.1  | <b>0.862</b> | 0.807        | 0.842        | 0.813        | 0.809        | 0.85         |
| PROMISE-Jedit-4.2  | <b>0.73</b>  | 0.65         | 0.615        | 0.685        | 0.712        | 0.727        |
| PROMISE-Log4j-1.0  | <b>0.889</b> | 0.875        | 0.8174       | 0.865        | 0.812        | 0.801        |
| PROMISE-Log4j-1.1  | <b>1</b>     | 0.954        | <b>1</b>     | 0.957        | 0.964        | 0.997        |
| PROMISE-Lucene-2.0 | 0.767        | 0.767        | 0.76         | 0.769        | <b>0.789</b> | 0.785        |
| PROMISE-Lucene-2.2 | 0.616        | 0.645        | <b>0.664</b> | 0.614        | 0.623        | 0.6125       |
| PROMISE-Tomcat     | <b>0.83</b>  | 0.799        | 0.802        | 0.772        | 0.825        | 0.809        |
| PROMISE-Xalan 2.4  | 0.749        | 0.804        | <b>0.849</b> | 0.816        | 0.7182       | 0.788        |
| PROMISE-Xalan 2.5  | 0.827        | 0.824        | 0.824        | 0.823        | <b>0.877</b> | 0.812        |
| PROMISE-Xalan-2.6  | <b>1</b>     | 0.988        | 0.998        | 0.991        | 0.962        | 0.99         |
| AEEEM-EQ           | 0.841        | <b>0.897</b> | 0.839        | 0.896        | 0.875        | 0.807        |
| AEEEM-JDT          | <b>0.881</b> | 0.814        | 0.881        | 0.805        | 0.803        | 0.8169       |
| AEEEM-LC           | <b>0.821</b> | 0.82         | 0.803        | 0.805        | 0.8033       | 0.805        |
| AEEEM-ML           | <b>0.838</b> | 0.822        | 0.837        | 0.801        | 0.811        | 0.808        |
| AEEEM-PDE          | 0.784        | 0.775        | 0.709        | 0.773        | <b>0.823</b> | 0.807        |
| NASA-Cm1           | <b>0.718</b> | 0.708        | 0.682        | 0.705        | 0.713        | 0.674        |
| NASA-Mw1           | 0.5          | 0.655        | 0.604        | 0.615        | 0.68         | <b>0.691</b> |
| NASA-PC1           | <b>0.925</b> | 0.888        | 0.894        | 0.905        | 0.893        | 0.817        |
| NASA-PC3           | 0.835        | 0.87         | <b>0.886</b> | 0.884        | 0.872        | 0.8176       |
| NASA-PC4           | <b>0.931</b> | 0.917        | 0.902        | 0.879        | 0.8919       | 0.852        |
| RELINK-Apache      | <b>0.697</b> | 0.655        | 0.604        | 0.615        | 0.618        | 0.691        |
| RELINK-Safe        | 0.786        | 0.737        | <b>0.817</b> | 0.785        | 0.802        | 0.816        |
| RELINK-Zxing       | 0.827        | 0.773        | 0.813        | <b>0.886</b> | 0.836        | 0.814        |

The significance of bold indicates the highest value for each dataset when performing the comparative analysis



## 5 Threats to validity

This section explores the potential threats to the validity of the study and outlines its limitations.

### 5.1 Internal validity

To ensure internal validity, we have used a modified version of the GWO algorithm, which incorporates a two-phase mutation operation to improve the algorithm's exploitation capability. We have used a sigmoid function to transform the continuous search space to a binary one, matching the binary nature of feature selection. We have also employed a two-phase mutation to reduce the number of selected features while preserving high classification accuracy.

### 5.2 External validity

The external validity pertains to the degree to which the findings of an empirical study can be generalized and applied beyond the specific conditions of the study. In this research, the proposed algorithm's performance was assessed across 27 datasets. This wide range of datasets ensures the generalizability of the results to various types of datasets and classification problems.

### 5.3 Construct validity

Construct validity ensures that both independent and dependent variables accurately represent the intended concepts. This paper does not encounter any threats to construct validity, as the predictors utilized in the study are commonly employed in predictive modelling within the domain of software engineering.

### 5.4 Conclusion validity

This concerns the degree to which the conclusions drawn in the study are robust. Statistical analyses were conducted to validate the efficacy of the proposed algorithm and its superiority. This encompassed comparative assessments between the proposed algorithm and other metaheuristic algorithms, with the results subjected to statistical scrutiny employing the Friedman Test and Wilcoxon Signed Rank Test.

The study is limited to the use of public/open-source datasets, which can lead to constraints in terms of the generalizability of the SDP model's findings to a broader range of software applications, particularly those in the industrial sector. Open-source projects typically exhibit

notable differences from proprietary or industrial projects in terms of development methodologies, team compositions, and code standards. These differences can impact the effectiveness of our proposed work when applied to industrial/proprietary software projects. In future, we aim to use the proposed SDP model for proprietary datasets from several industries to evaluate its predictive performance and improve the model for wider applicability.

## 6 Conclusion and future work

In this study, we proposed a prediction model based on 2M-GWO for SDP, which incorporates FS to enhance the quality of the ML techniques, i.e., SVM, AB, GB, NB and KNN, are used for the classification of the binary class dataset. We systematically designed the experiments based on 27 datasets from AEEEM, PROMISE, RELINK and NASA datasets. The key findings of the study are as follows:

- The proposed two-phase Grey Wolf Optimizer-based feature selection method, paired with the Random Forest classifier (2M-GWO-RF), effectively enhances software defect prediction (SDP). By reducing feature size, the 2M-GWO method improves model performance across multiple datasets and supports the accurate identification and removal of redundant and irrelevant features.
- The 2M-GWO-RF model achieves superior results, ranking highest in terms of ROC\_AUC with a mean rank of 2.19, demonstrating exceptional prediction accuracy and efficiency (RQ2).
- Statistical analysis using Friedman's test confirms the significance of the mean rank differences ( $p = 0.034$ ), validating 2M-GWO-RF as the top classification technique.
- The proposed model's robustness is evident through its focus on key metrics across various datasets, including CountLineCodeDecl, rfc, ce, npm, CVSEntropy, and LOC CODE AND COMMENT, among others, indicating its potential for future data collection and analysis.
- The 2M-GWO-RF model outperforms five related metaheuristic techniques in over 55% of the datasets, showcasing its superior choice in comparison to state-of-the-art approaches.

The study concludes that the two-phase Grey Wolf Optimizer with Random Forest classifier (2M-GWO-RF) significantly enhances software defect prediction by



offering superior feature selection and high prediction capability.

**Acknowledgements** Not applicable.

**Author contributions** RM and KK jointly contributed to the conceptualization and design of the study. KK undertook the data collection and analysis process. KK drafted the initial manuscript, which was subsequently reviewed and commented on by RM. All authors participated in the review process and provided final approval for the manuscript.

**Funding** The authors declare that no funds, grants, or other support were received for the conduct of this study and preparation of this manuscript.

**Data availability** The PROMISE, RELINK, NASA and AEEEM datasets used in this study are publicly available at (<https://zenodo.org/records/3693686>), (<https://www.cse.ust.hk/~scc/RELINK.htm>), (<https://github.com/klainfo/NASADefectDataset>) and (<https://bug.inf.usi.ch/download.php>) respectively.

## Declarations

**Competing interests** The authors declare no competing interests.

**Ethical approval** Not applicable.

**Human participants and/or animals** Not applicable.

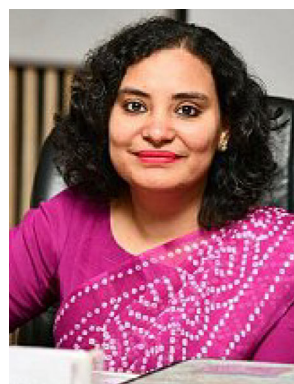
## References

- Ramulu, K., Murhtyr, B.V.R.: Importance of software quality models in software engineering. *Int. J. Eng. Technol. Manag. Res.* **5**, 200–218 (2020). <https://doi.org/10.29121/ijetmr.v5.i3.2018.192>
- Miguel, J.P., Mauricio, D., Rodríguez, G.: A review of software quality models for the evaluation of software products. *Int. J. Softw. Eng. Appl.* **5**, 31–53 (2014). <https://doi.org/10.5121/ijsea.2014.5603>
- Yu, Q., Jiang, S., Qian, J.: Which is more important for cross-project defect prediction: instance or feature? In: 2016 International Conference on Software Analysis, Testing and Evolution (SATE). IEEE, pp. 90–95 (2016)
- Balogun, A.O., Basri, S., Mahamad, S., et al.: Impact of feature selection methods on the predictive performance of software defect prediction models: an extensive empirical study. *Symmetry* **12**, 1147 (2020). <https://doi.org/10.3390/sym12071147>
- Kakkar, M., Jain, S.: Feature selection in software defect prediction: a comparative study. In: 2016 6th International Conference—Cloud System and Big Data Engineering (Confluence). IEEE, pp 658–663 (2016)
- Abdel-Basset, M., El-Shahat, D., El-henawy, I., et al.: A new fusion of grey wolf optimizer algorithm with a two-phase mutation for feature selection. *Expert Syst. Appl.* **139**, 112824 (2020). <https://doi.org/10.1016/j.eswa.2019.112824>
- Kassaymeh, S., Abdullah, S., Al-Betar, M.A., Alweshah, M.: Salp swarm optimizer for modeling the software fault prediction problem. *J. King Saud Univ.-Comput. Inf. Sci.* **34**, 3365–3378 (2022). <https://doi.org/10.1016/j.jksuci.2021.01.015>
- Ezugwu, A.E., Shukla, A.K., Nath, R., et al.: Metaheuristics: a comprehensive overview and classification along with bibliometric analysis. *Artif. Intell. Rev.* **54**, 4237–4316 (2021). <https://doi.org/10.1007/s10462-020-09952-0>
- Ferenc, R., Toth, Z., Ladanyi, G., et al.: A public unified bug dataset for java and its assessment regarding metrics and bug prediction. *Softw. Qual. J.* **28**, 1447–1506 (2020). <https://doi.org/10.1007/s11219-020-09515-0>
- Jureczko, M., Madeyski, L.: Towards identifying software project clusters with regard to defect prediction. In: Proceedings of the 6th International Conference on Predictive Models in Software Engineering. ACM, pp. 1–10 (2010)
- Wu, R., Zhang, H., Kim, S., Cheung, S.-C.: RELINK: recovering links between bugs and changes. In: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering, pp. 15–25 (2011)
- D'Ambros, M., Lanza, M., Robbes, R.: Evaluating defect prediction approaches: a benchmark and an extensive comparison. *Empir. Softw. Eng.* **17**, 531–577 (2012). <https://doi.org/10.1007/s10664-011-9173-9>
- Mirjalili, S., Mirjalili, S.M., Lewis, A.: Grey wolf optimizer. *Adv. Eng. Softw.* **69**, 46–61 (2014). <https://doi.org/10.1016/j.advengsoft.2013.12.007>
- El-Hasnony, I.M., Barakat, S.I., Mostafa, R.R.: Optimized ANFIS model using hybrid metaheuristic algorithms for Parkinson's disease prediction in IoT environment. *IEEE Access* **8**, 119252–119270 (2020). <https://doi.org/10.1109/ACCESS.2020.3005614>
- Mirjalili, S., Lewis, A.: S-shaped versus V-shaped transfer functions for binary Particle Swarm Optimization. *Swarm Evol. Comput.* **9**, 1–14 (2013). <https://doi.org/10.1016/j.swevo.2012.09.002>
- Goyal, S.: Genetic evolution-based feature selection for software defect prediction using SVMs. *J. Circ. Syst. Comput.* **31**, 2250161 (2022). <https://doi.org/10.1142/S0218126622501614>
- Breiman, L.: Random forests. *Mach. Learn.* **45**, 5–32 (2001)
- Gezici, B., Tarhan, A.K.: Explainable AI for software defect prediction with gradient boosting classifier. In: 2022 7th International Conference on Computer Science and Engineering (UBMK), pp. 1–6 (2022)
- Karim, S., Leslie Hendric Spits Warnars, H., Lumban Gaol, F., et al.: Software metrics for fault prediction using machine learning approaches a literature review with PROMISE repository dataset. In: 2017 IEEE International Conference on Cybernetics and Computational Intelligence (Cybernetics Com), pp. 19–23 (2017)
- Dahim, M., Alqadhi, S., Mallick, J.: Enhancing landslide management with hyper-tuned machine learning and deep learning models: predicting susceptibility and analyzing sensitivity and uncertainty. *Front. Ecol. Evol.* **11**, 1108924 (2023). <https://doi.org/10.3389/fevo.2023.1108924>
- Malik, N.U.R., Sheikh, U.U., Abu-Bakar, S.A.R., Channa, A.: Multi-view human action recognition using skeleton based-FineKNN with extraneous frame scrapping technique. *Sensors* **23**, 2745 (2023). <https://doi.org/10.3390/s23052745>
- Malhotra, R., Khan, K.: A study on software defect prediction using feature extraction techniques. In: 2020 8th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO). IEEE, pp. 1139–1144 (2020)
- Heidari, A.A., Mirjalili, S., Faris, H., et al.: Harris hawks optimization: Algorithm and applications. *Future Gener. Comput. Syst.* **97**, 849–872 (2019). <https://doi.org/10.1016/j.future.2019.02.028>
- Khatri, Y., Singh, S.K.: An effective feature selection based cross-project defect prediction model for software quality improvement. *Int. J. Syst. Assur. Eng. Manag.* **14**, 154–172 (2023). <https://doi.org/10.1007/s13198-022-01831-x>

25. Mirjalili, S., Lewis, A.: The whale optimization algorithm. *Adv. Eng. Softw.* **95**, 51–67 (2016). <https://doi.org/10.1016/j.advengsoft.2016.01.008>
26. da Silva, L.S.A., Lúcio, Y.L.S., dos Coelho, L.S., et al.: A comprehensive review on Jaya optimization algorithm. *Artif. Intell. Rev.* **56**, 4329–4361 (2023). <https://doi.org/10.1007/s10462-022-10234-0>
27. Sharma, T., Sangwan, O.P.: Sine-Cosine algorithm for software fault prediction. In: *Proceedings—2021 IEEE International Conference on Software Maintenance and Evolution, ICSME 2021*. Institute of Electrical and Electronics Engineers Inc., pp. 701–706 (2021)
28. Riaz, S., Arshad, A., Jiao, L.: Rough noise-filtered easy ensemble for software fault prediction. *IEEE Access* **6**, 46886–46899 (2018). <https://doi.org/10.1109/ACCESS.2018.2865383>
29. Tumar, I., Hassouneh, Y., Turabieh, H., Thaher, T.: Enhanced binary moth flame optimization as a feature selection algorithm to predict software fault prediction. *IEEE Access* **8**, 8041–8055 (2020). <https://doi.org/10.1109/ACCESS.2020.2964321>
30. Brundha Elci, J., Nandagopalan, S.: SS-WDRN: sparrow search optimization-based weighted dual recurrent network for software fault prediction. *Knowl. Inf. Syst.* **66**, 1037–1064 (2024). <https://doi.org/10.1007/s10115-023-01932-4>
31. Sekaran, K., Lawrence, S.P.A.: Mutation boosted salp swarm optimizer meets rough set theory: a novel approach to software defect detection. *Trans. Emerg. Telecommun. Technol.* (2024). <https://doi.org/10.1002/ett.4953>
32. Das, H., Prajapati, S., Gourisaria, M.K., et al.: Feature selection using golden jackal optimization for software fault prediction. *Mathematics* **11**, 2438 (2023). <https://doi.org/10.3390/math11112438>
33. Tang, Y., Dai, Q., Yang, M., et al.: Software defect prediction ensemble learning algorithm based on adaptive variable sparrow search algorithm. *Int. J. Mach. Learn. Cybern.* (2023). <https://doi.org/10.1007/s13042-022-01740-2>
34. Mafarja, M., Thaher, T., Al-Betar, M.A., et al.: Classification framework for faulty-software using enhanced exploratory whale optimizer-based feature selection scheme and random forest ensemble learning. *Appl. Intell.* **53**, 18715–18757 (2023). <https://doi.org/10.1007/s10489-022-04427-x>
35. Zada, I., Alshammari, A., Mazhar, A.A., et al.: Enhancing IOT based software defect prediction in analytical data management using war strategy optimization and Kernel ELM. *Wirel. Netw.* (2023). <https://doi.org/10.1007/s11276-023-03591-3>
36. Alweshah, M., Kassaymeh, S., Alkhalailah, S., et al.: An efficient hybrid mine blast algorithm for tackling software fault prediction problem. *Neural. Process. Lett.* **55**, 10925–10950 (2023). <https://doi.org/10.1007/s11063-023-11357-3>
37. Gupta, A., Sharma, M., Srivastava, A.: A novel dimensionality reduction-based software bug prediction using a bat-inspired algorithm. In: *Proceedings of the 13th International Conference on Cloud Computing, Data Science and Engineering, Confluence 2023*. Institute of Electrical and Electronics Engineers Inc., pp. 278–285 (2023)
38. Sahu, S.P., Reddy, B.R., Mukherjee, D., et al.: A hybrid approach to software fault prediction using genetic programming and ensemble learning methods. *Int. J. Syst. Assur. Eng. Manag.* **13**, 1746–1760 (2022). <https://doi.org/10.1007/s13198-021-01532-x>
39. Balaram, A., Vasundra, S.: Prediction of software fault-prone classes using ensemble random forest with adaptive synthetic sampling algorithm. *Autom. Softw. Eng.* **29**, 6 (2022). <https://doi.org/10.1007/s10515-021-00311-z>
40. Thirumoorthy, K., Jerold John Britto, J.: A feature selection model for software defect prediction using binary Rao optimization algorithm. *Appl. Soft Comput.* **131**, 109737 (2022). <https://doi.org/10.1016/j.asoc.2022.109737>
41. Thirumoorthy, K., Britto, J.J.J.: A clustering approach for software defect prediction using hybrid social mimic optimization algorithm. *Computing* **104**, 2605–2633 (2022). <https://doi.org/10.1007/s00607-022-01100-6>
42. Zhu, K., Ying, S., Zhang, N., Zhu, D.: Software defect prediction based on enhanced metaheuristic feature selection optimization and a hybrid deep neural network. *J. Syst. Softw.* **180**, 111026 (2021). <https://doi.org/10.1016/j.jss.2021.111026>
43. Hassouneh, Y., Turabieh, H., Thaher, T., et al.: Boosted whale optimization algorithm with natural selection operators for software fault prediction. *IEEE Access* **9**, 14239–14258 (2021). <https://doi.org/10.1109/ACCESS.2021.3052149>
44. Thaher, T., Arman, N.: Efficient multi-swarm binary harris hawks optimization as a feature selection approach for software fault prediction. In: *2020 11th International Conference on Information and Communication Systems, ICICS 2020*. Institute of Electrical and Electronics Engineers Inc., pp. 249–254 (2020)
45. Kalavani, N., Beena, R.: Boosted relief feature subset selection and heterogeneous cross project defect prediction using firefly particle swarm optimization. *Int. J. Recent Technol. Eng.* **8**, 2605–2613 (2020). <https://doi.org/10.35940/ijrte.E6333.018520>
46. Anbu, M., Anandha Mala, G.S.: Feature selection using firefly algorithm in software defect prediction. *Clust. Comput.* **22**, 10925–10934 (2019). <https://doi.org/10.1007/s10586-017-1235-3>
47. Turabieh, H., Mafarja, M., Li, X.: Iterated feature selection algorithms with layered recurrent neural network for software fault prediction. *Expert Syst. Appl.* **122**, 27–42 (2019). <https://doi.org/10.1016/j.eswa.2018.12.033>
48. Khuat, T.T., Le, M.H.: Binary teaching–learning-based optimization algorithm with a new update mechanism for sample subset optimization in software defect prediction. *Soft Comput.* **23**, 9919–9935 (2019). <https://doi.org/10.1007/s00500-018-3546-6>

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



**Ruchika Malhotra** is the Head of the Department and a Professor in the Department of Software Engineering at Delhi Technological University (DTU), Delhi, India. She has previously served as the Associate Dean of Industrial Research and Development at DTU. She was awarded the prestigious Raman Fellowship for post-doctoral research at Indiana University-Purdue University Indianapolis, USA. She earned her master's and doctorate degrees in software engineering from the University School of Information Technology at Guru Gobind Singh Indraprastha University, Delhi, India. In 2013, she received the IBM Faculty Award. Her contributions to the field have earned her recognition as one of the world's top 2% scientists, according to a Stanford University report in 2020, specifically for her work in "Artificial Intelligence & Image Processing" (PLOS

Biology, 2020). She has also received the Commendable Research Award from Delhi Technological University for the years 2018-2022. Her h-index is 37 as reported by Google Scholar. She is the author of the book “Empirical Research in Software Engineering,” published by CRC Press, and co-author of “Object Oriented Software Engineering,” published by PHI Learning. She has published over 200 research papers in international journals and conferences. Her research interests include software testing, software quality improvement, statistical and adaptive prediction models, software metrics, and the definition and validation of software metrics.



**Kishwar Khan** received her B.Tech. degree in Computer Science and Engineering from Dr. A. P. J. Abdul Kalam Technical University, Lucknow, India, and her M.Tech. degree in Software Engineering from Delhi Technological University, Delhi, India. She is currently pursuing a Ph.D. in Software Engineering at Delhi Technological University. Her doctoral research focuses on software quality improvement and the application of machine learning

and deep learning in various aspects of software quality. Her current research interests include Artificial Intelligence, software quality, predictive modelling, and data analytics.